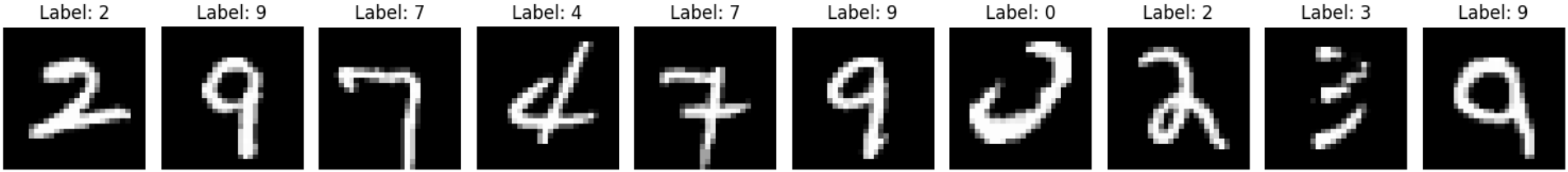
Introduction to Deep Learning

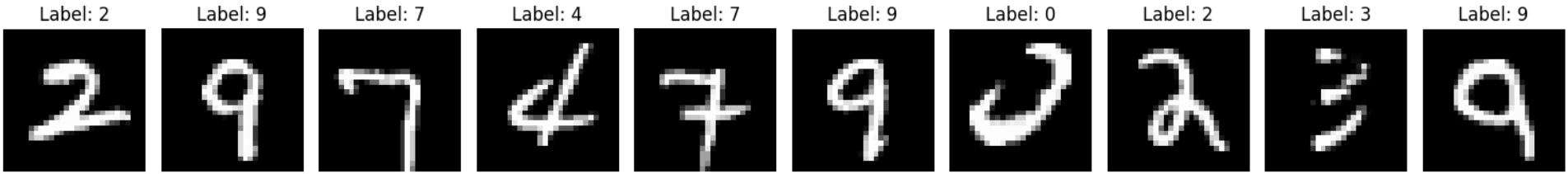
CS/EE/CMS 148a - Winter 2026

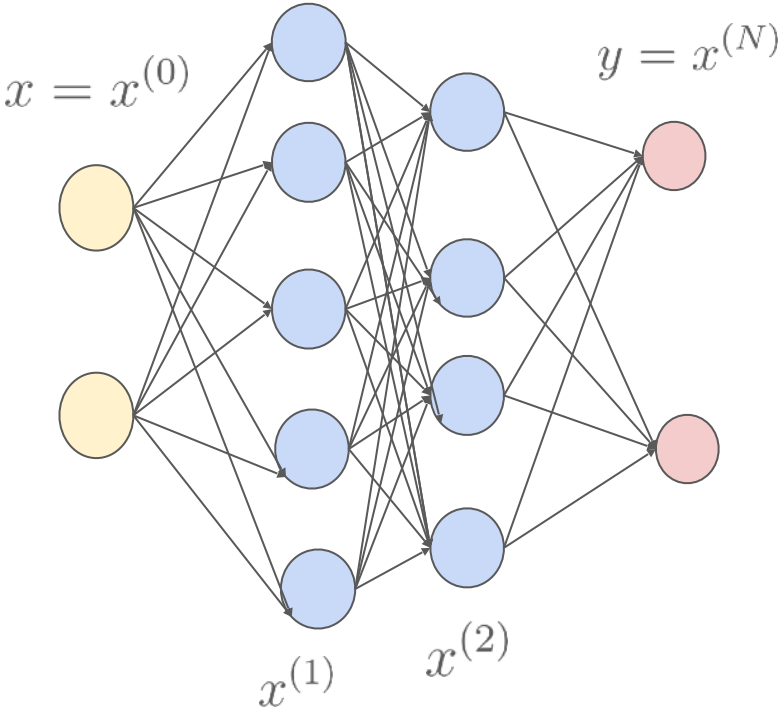
Lecture 6 - Training, good fit and generalization

Training Neural Networks



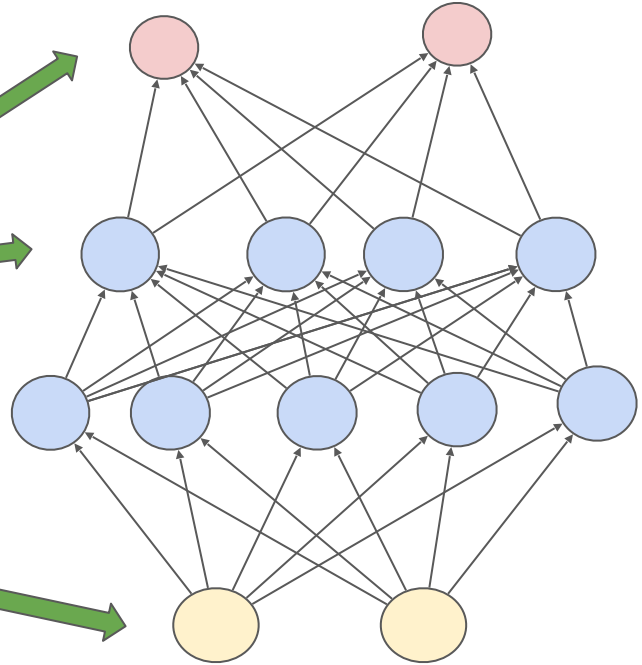
Deng, L. (2012). The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, *29*(6), 141–142.



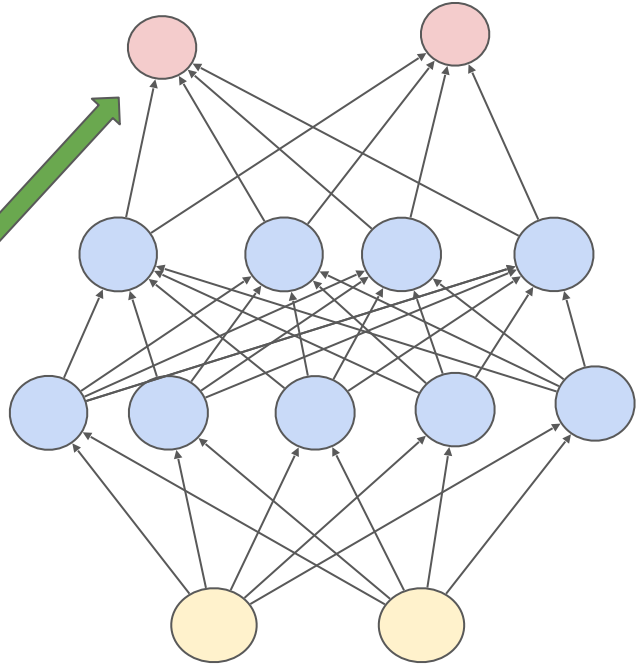


Deng, L. (2012). The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, *29*(6), 141–142.

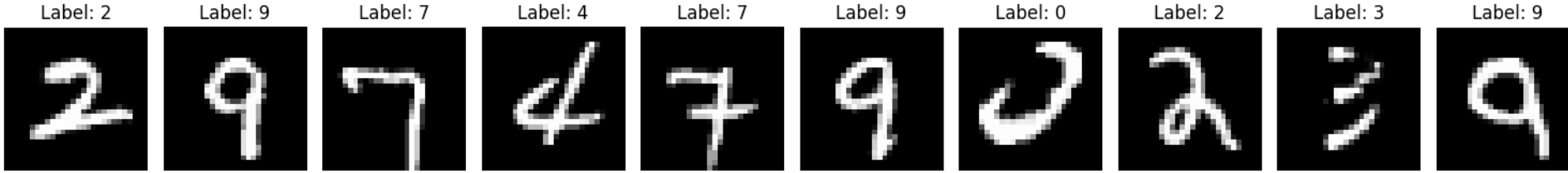
```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

# Define the perceptron model
class Perceptron(nn.Module):
    def __init__(self):
        super(Perceptron, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
```



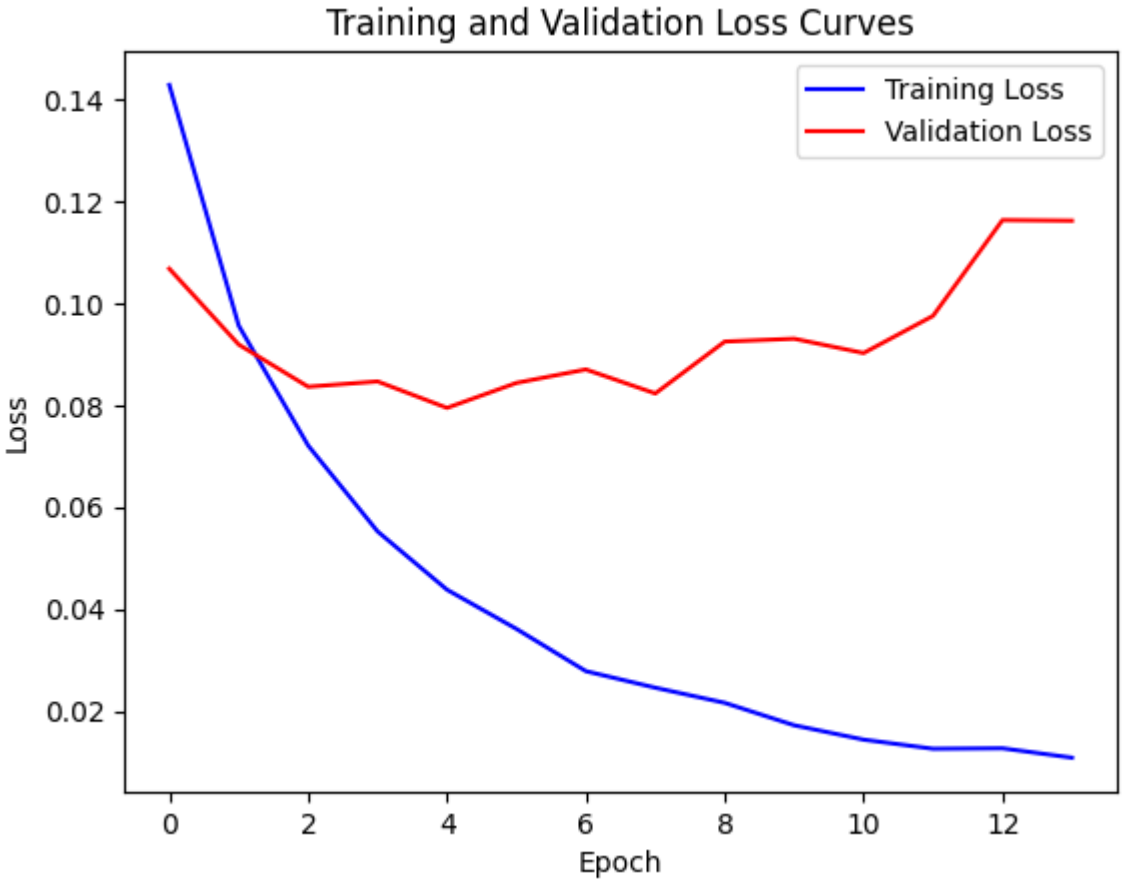
```python
…
# Define the perceptron model
class Perceptron(nn.Module):
    def __init__(self):
        super(Perceptron, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x
# Create the model, optimizer, and loss function
model = Perceptron()
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

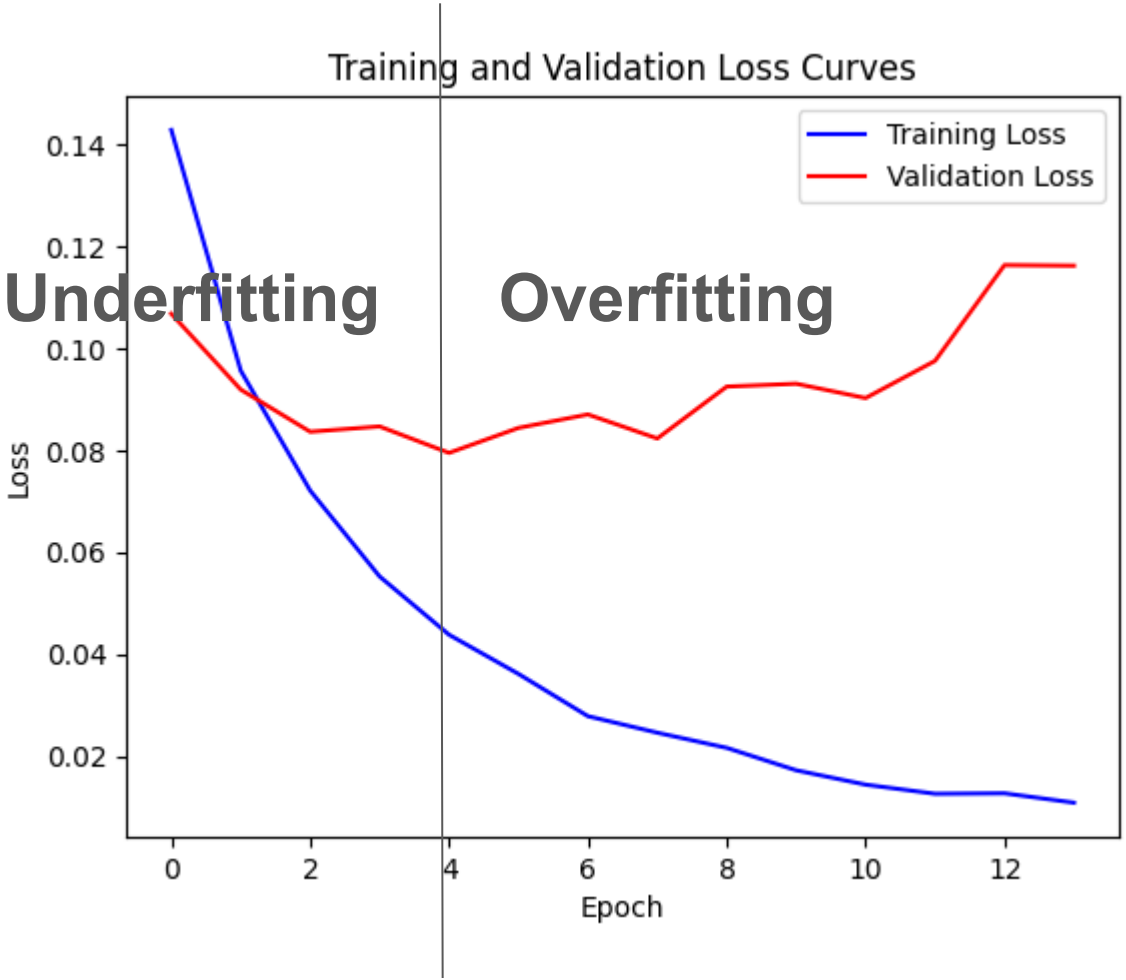


```python
# Load MNIST dataset
train_dataset = datasets.MNIST('./data', train=True, download=True,
                  transform=transforms.ToTensor())
test_dataset = datasets.MNIST('./data', train=False, download=True,
                 transform=transforms.ToTensor())

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)
```



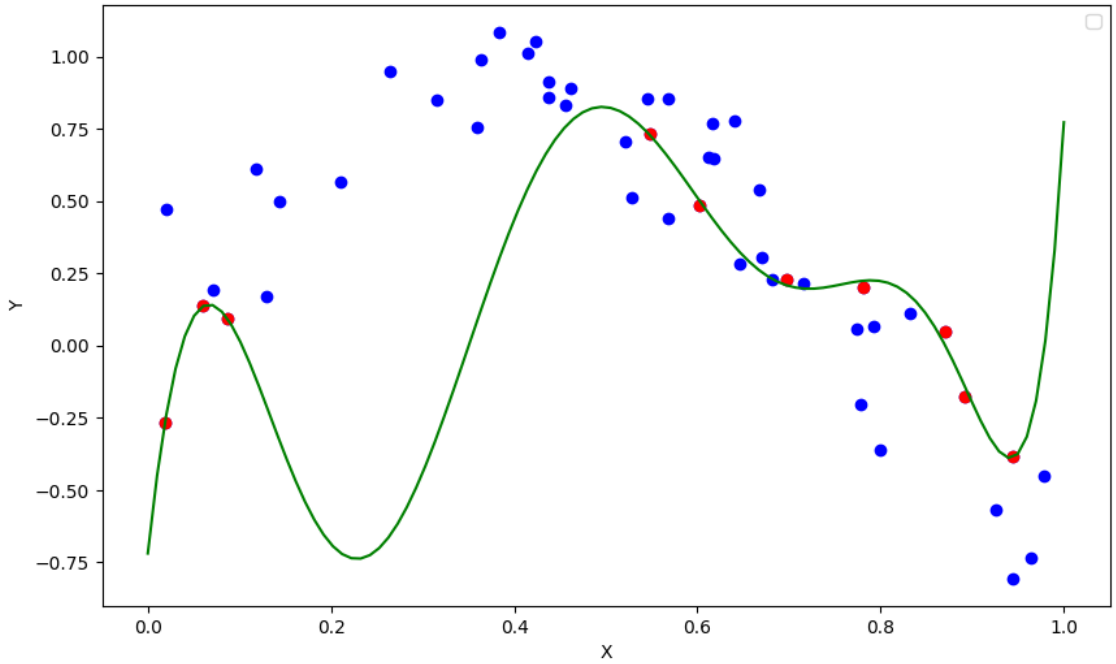
```python
# Training loop
epochs = 15
train_losses = []
val_losses = []
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    train_losses.append(running_loss / len(train_loader))
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)
            loss = criterion(output, target)
            val_loss += loss.item()
    val_losses.append(val_loss / len(test_loader))
```


Training and Validation Loss Curves
Training Loss
Validation Loss
Loss
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0
2
4
6
8
10
12
Epoch


Training and Validation Loss Curves
Training Loss
Validation Loss
Underfitting
Overfitting
Loss
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0
2
4
6
8
10
12
Epoch

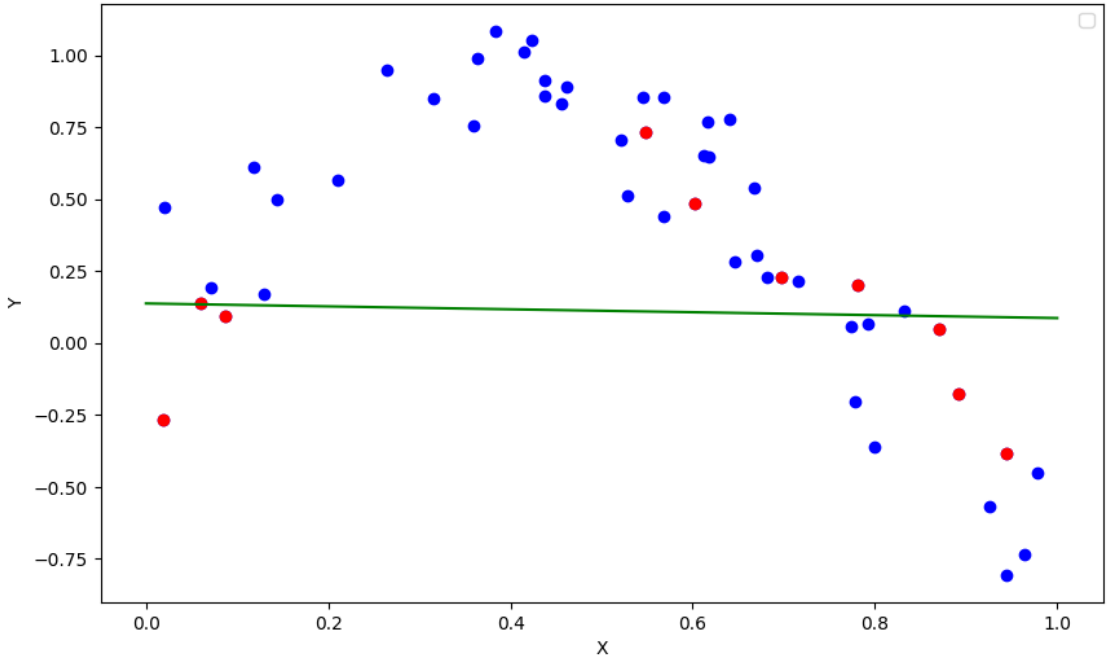
Overfitting

The model fits the training data too well and cannot generalize



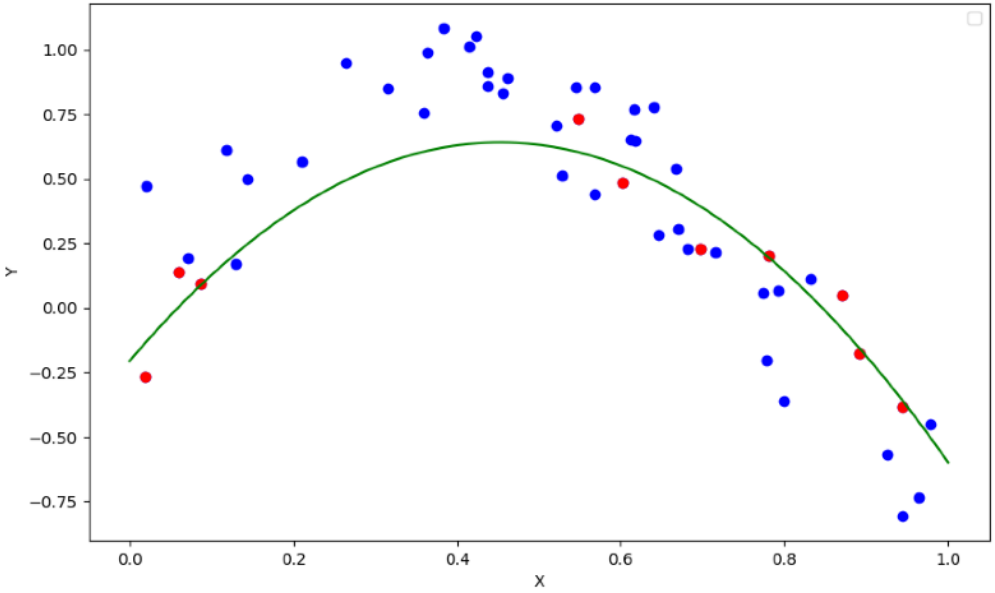
Underfitting

The model does not have enough capacity to fit the data



Inductive bias

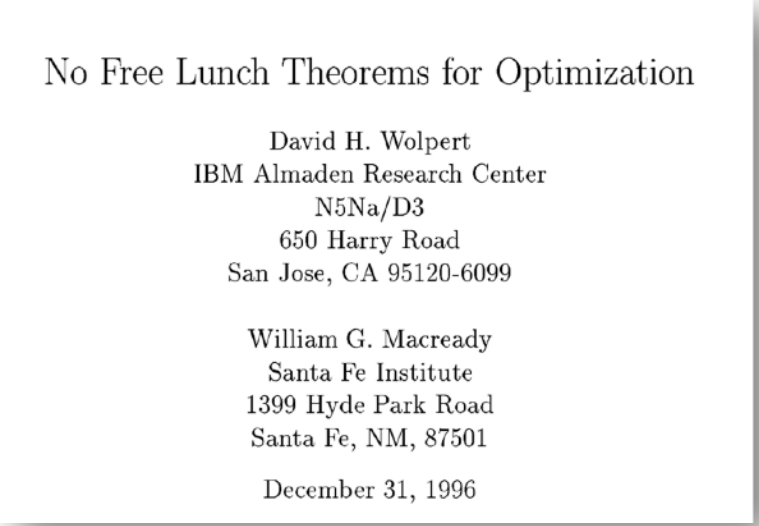
Set of constraints on class of function, parameters, or training algorithm, to prevent all of the above.



No free lunch

(Wolpert and Macready, 1996)

Any two optimization algorithms are equivalent when their performance is averaged across all possible problems.



No Free Lunch Theorems for Optimization

David H. Wolpert
IBM Almaden Research Center
N5Na/D3
650 Harry Road
San Jose, CA 95120-6099

William G. Macready
Santa Fe Institute
1399 Hyde Park Road
Santa Fe, NM, 87501

December 31, 1996

No free lunch

(Wolpert and Macready, 1997)

Any two optimization algorithms are equivalent when their performance is averaged across all possible problems.

That is, for every problem we will have to make appropriate tradeoffs between bias and variance,

No free lunch

(Wolpert and Macready, 1997)

Any two optimization algorithms are equivalent when their performance is averaged across all possible problems.

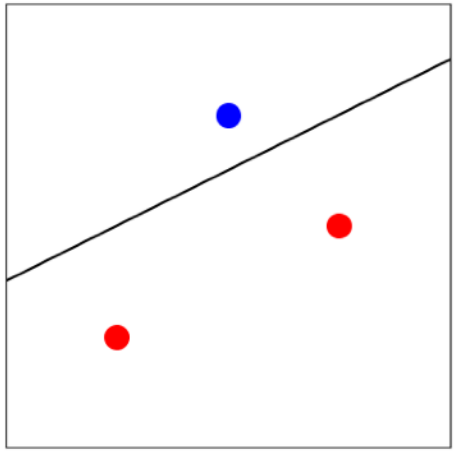
That is, for every problem we will have to make appropriate tradeoffs between bias and variance, and chose the

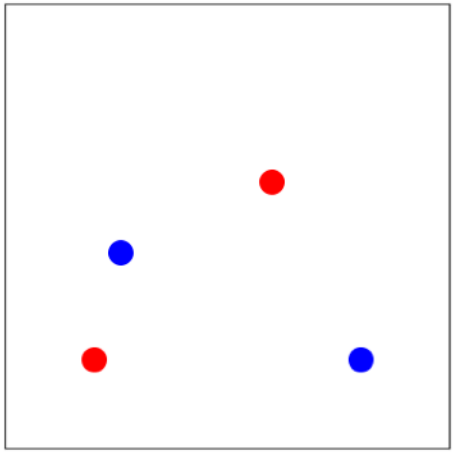
- class of functions
- error function
- parameters constraints
- training algorithm
- data handling

that is most appropriate. Most likely, a combination of those!

How to measure

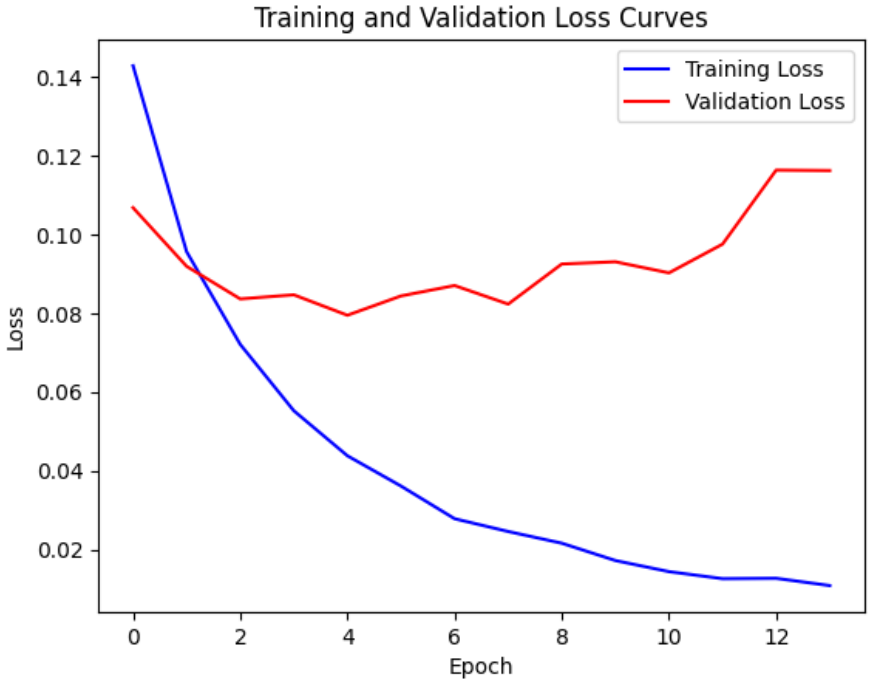
VC (Vapnik–Chervonenkis) dimension (~1970) : the largest set of points that a function can shatter (distinguish any labeling of those points)



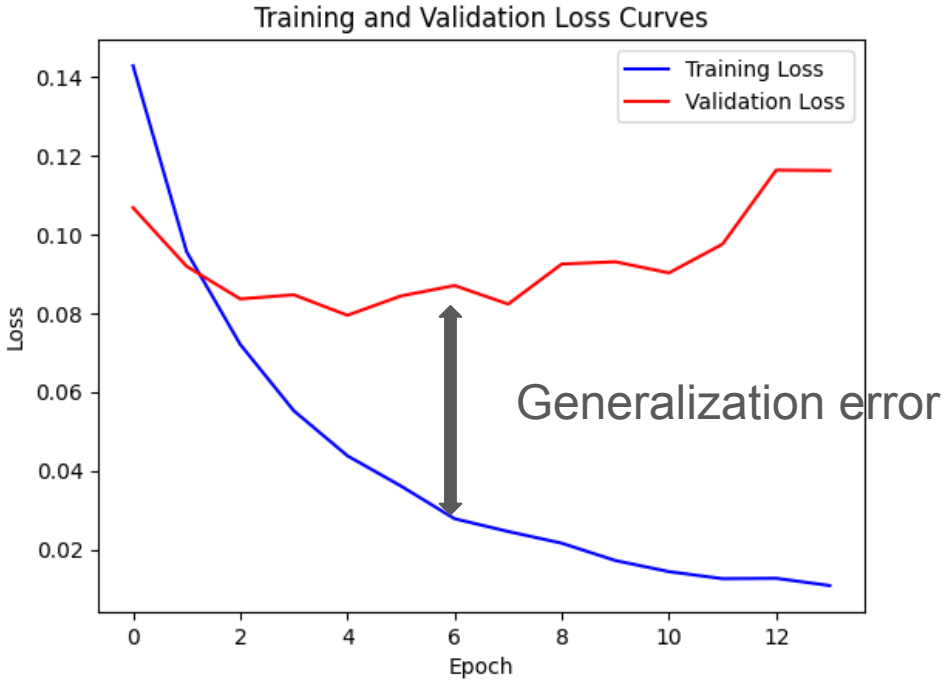


Linear functions in the plane: VC=3

VC is not a great approach for Deep Neural networks: it is very hard to calculate.



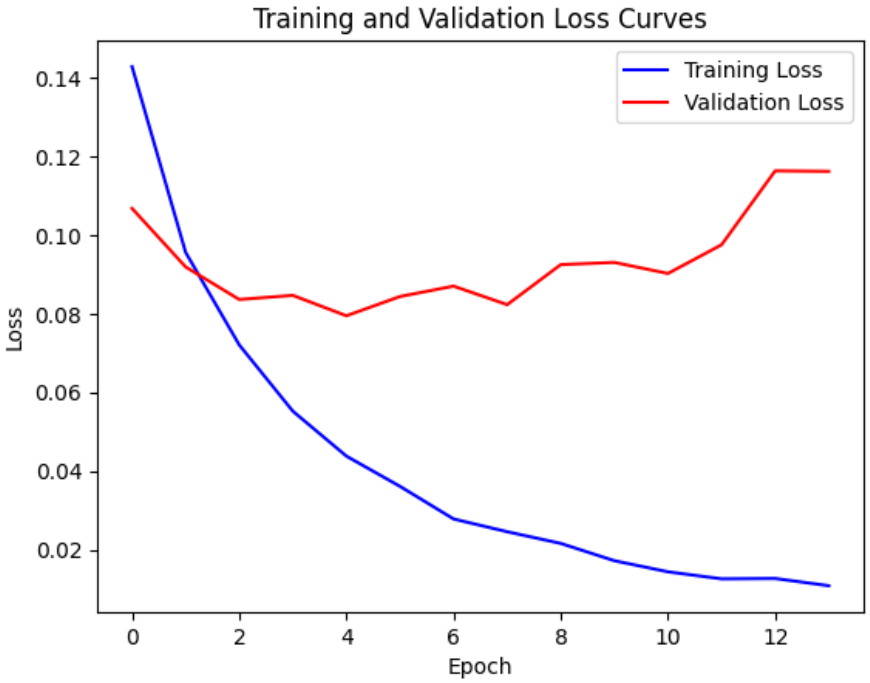
VC is not a great approach for Deep Neural networks: it is very hard to calculate.
The preferred approach is to measure generalization error.



Class of functions

Class of functions

Deep networks are a very large class of functions. You always end up overfitting! Choice of the architecture will help. This will be the topic of the next section of the class.



Invariance vs equivariance

What? Where? Shape?

Invariance vs equivariance

	What?	Where?	Shape?
	Bird		

Invariance vs equivariance

	What?	Where?	Shape?
	Bird		
	Bird		

Invariance vs equivariance

	What?	Where?	Shape?
	Bird		
	Bird		
	Bird		

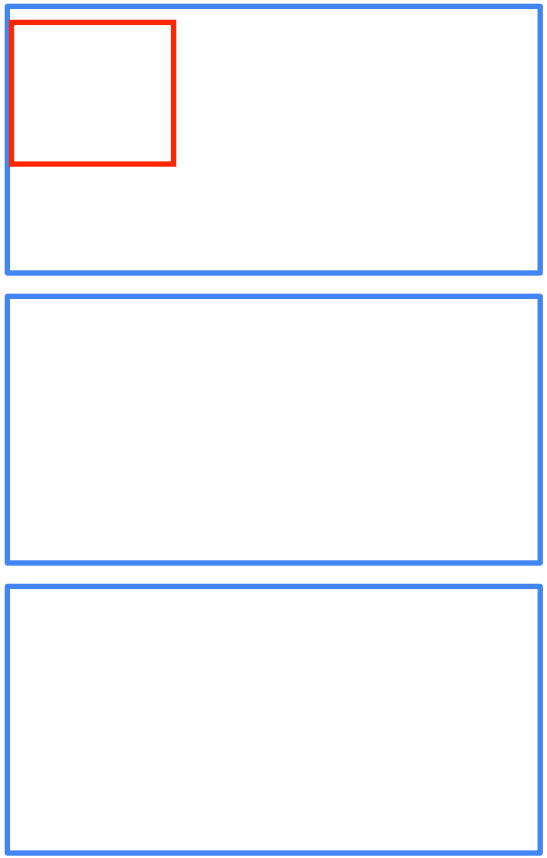
Invariance vs equivariance

What? Where? Shape?

Bird

Bird

Bird



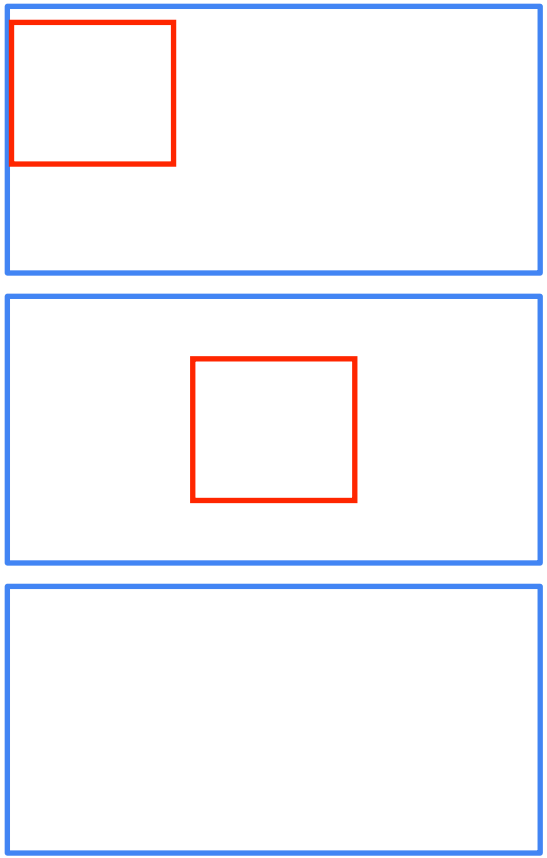
Invariance vs equivariance

What? Where? Shape?

Bird

Bird

Bird



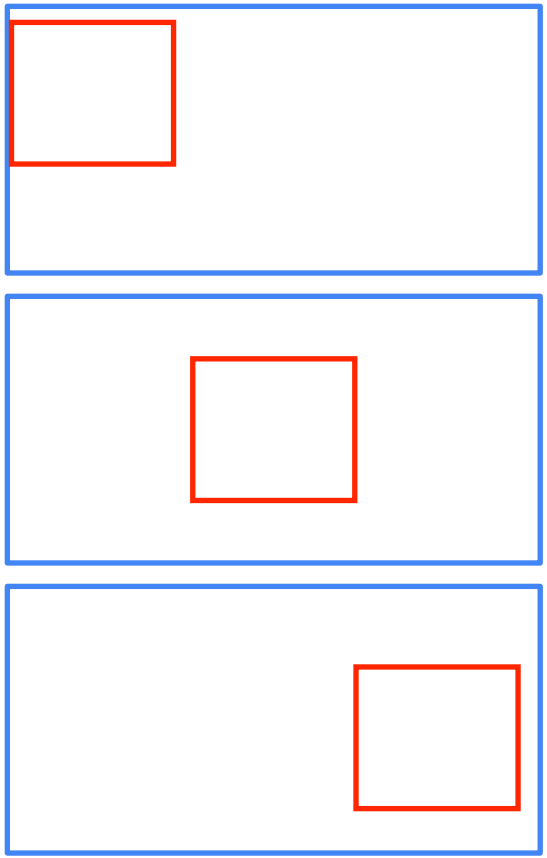
Invariance vs equivariance

What? Where? Shape?

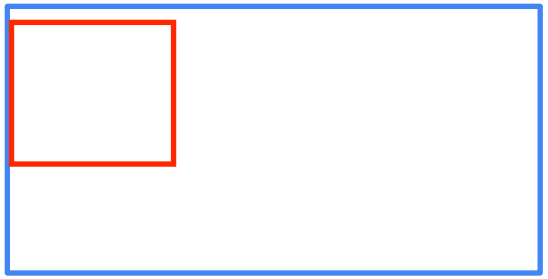
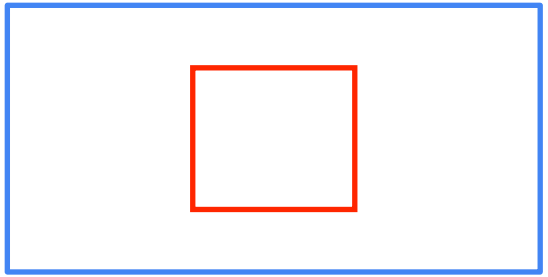
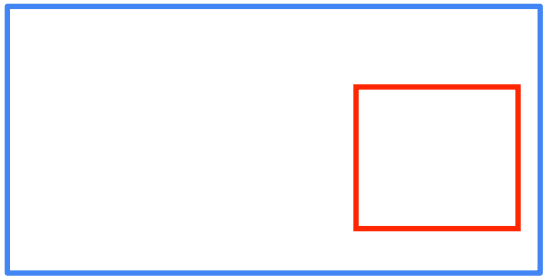
Bird

Bird

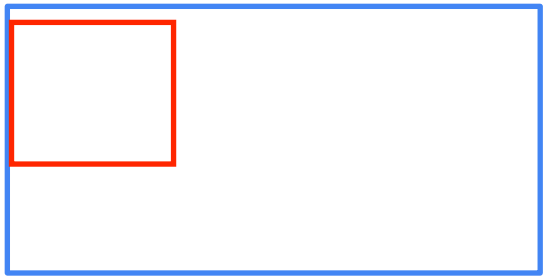
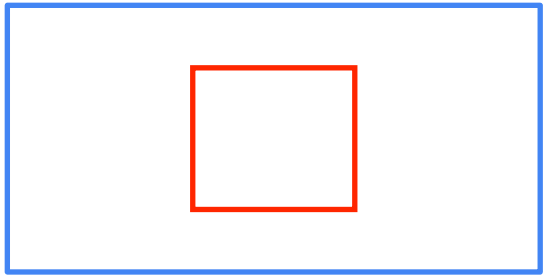
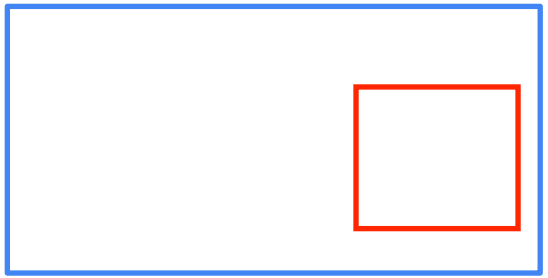
Bird



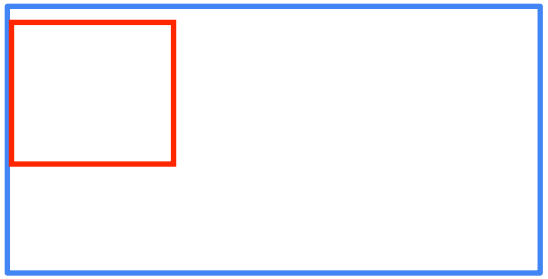
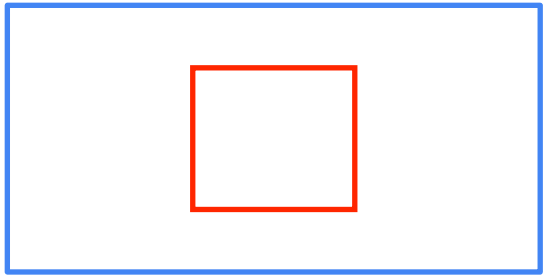
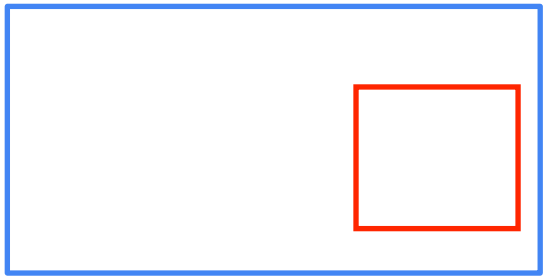
Invariance vs equivariance

	What?	Where?	Shape?
	Bird		4:3
	Bird		
	Bird		

Invariance vs equivariance

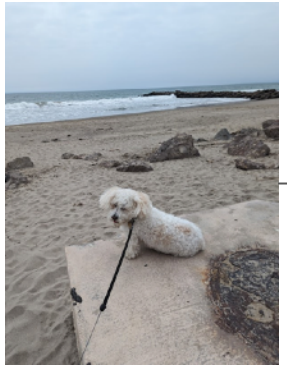
	What?	Where?	Shape?
	Bird		4:3
	Bird		4:3
	Bird		

Invariance vs equivariance

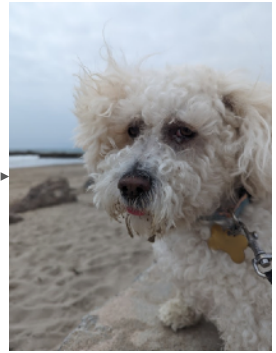
	What?	Where?	Shape?
	Bird		4:3
	Bird		4:3
	Bird		4:3

Invariance and equivariance

Implicit way to constrain a class of function

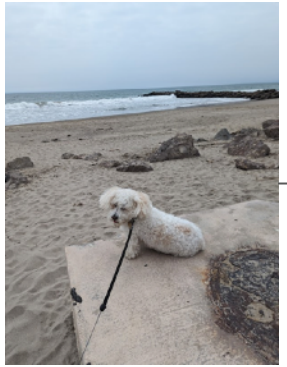


Zoom in

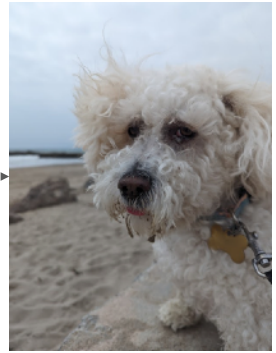


Invariance and equivariance

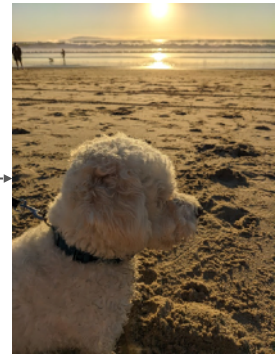
Implicit way to constrain a class of function



Zoom in



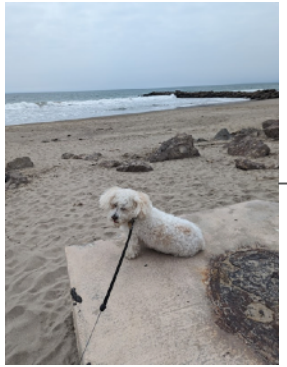
Change pose



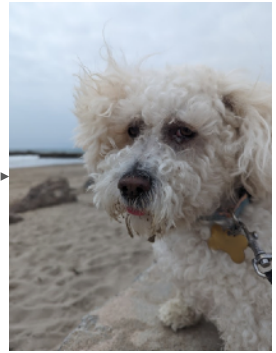
Always a dog!

Invariance and equivariance

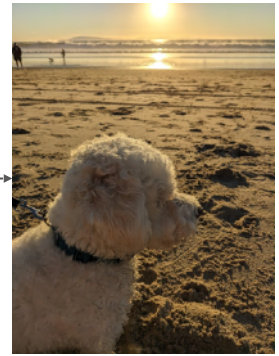
Implicit way to constrain a class of function



Zoom in



Change pose



Move

Always a dog!

Class of function - invariance and equivariance

Implicit way to constrain a class of function

How to impose such constraints on neural networks?

- Invariant features - although most times NN learn those by themselves

Class of function - invariance and equivariance

Implicit way to constrain a class of function

How to impose such constraints on neural networks?

- Invariant features - although most times NN learn those by themselves
- Architecture - e.g. weight tying (convolutions)

Class of function - invariance and equivariance

Implicit way to constrain a class of function

How to impose such constraints on neural networks?

- Invariant features - although most times NN learn those by themselves
- Architecture - e.g. weight tying (convolutions)
- Data augmentation - apply invariance

Class of function - invariance and equivariance

Implicit way to constrain a class of function

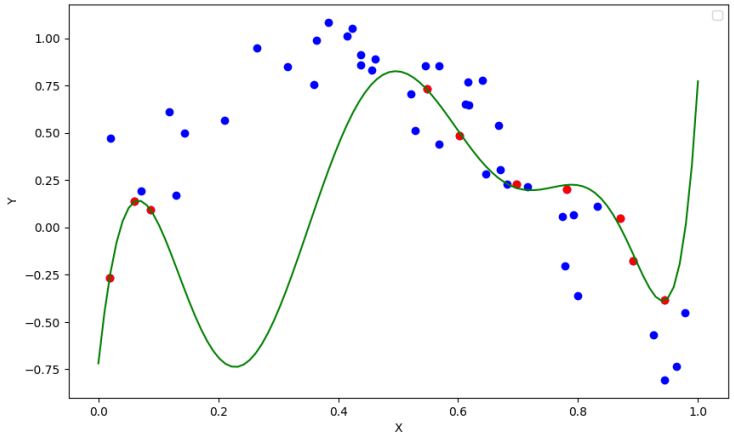
How to impose such constraints on neural networks?

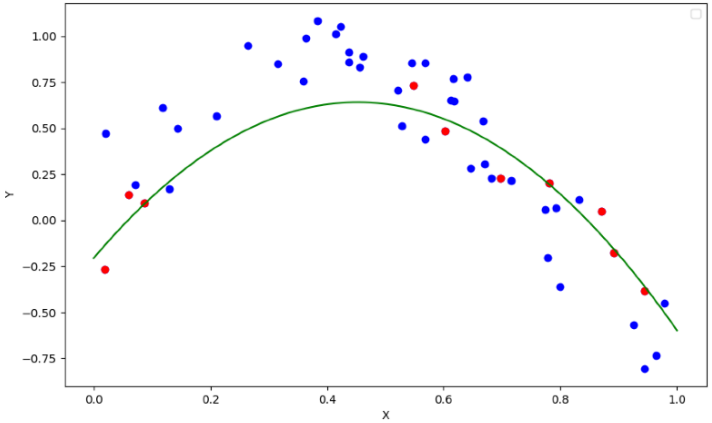
- Invariant features - although most times NN learn those by themselves
- Architecture - e.g. weight tying (convolutions)
- Data augmentation - apply invariance
- Regularized error - add an error component to penalize when the invariants are not respected

Error function and constraints on parameters

Regularization

Smoothness helps to generalize





Regularization through weight decay

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

Regularization through weight decay

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

L1 : promotes sparse weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

Regularization through weight decay

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

L1 : promotes sparse weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

L2 : promotes small weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$

Regularization through weight decay

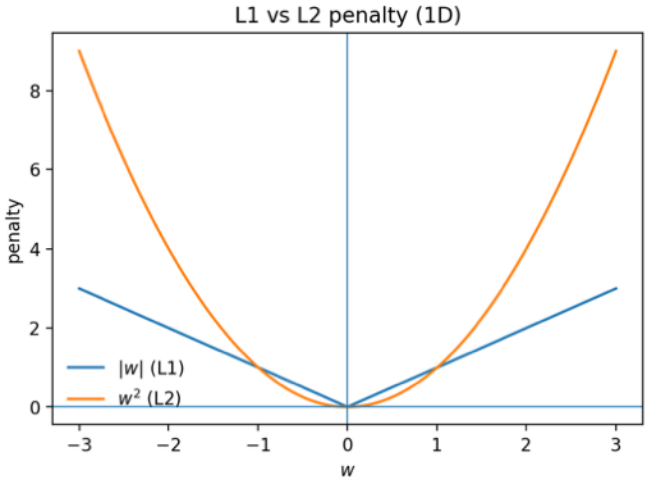
$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

L1 : promotes sparse weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

L2 : promotes small weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$



Regularization through weight decay

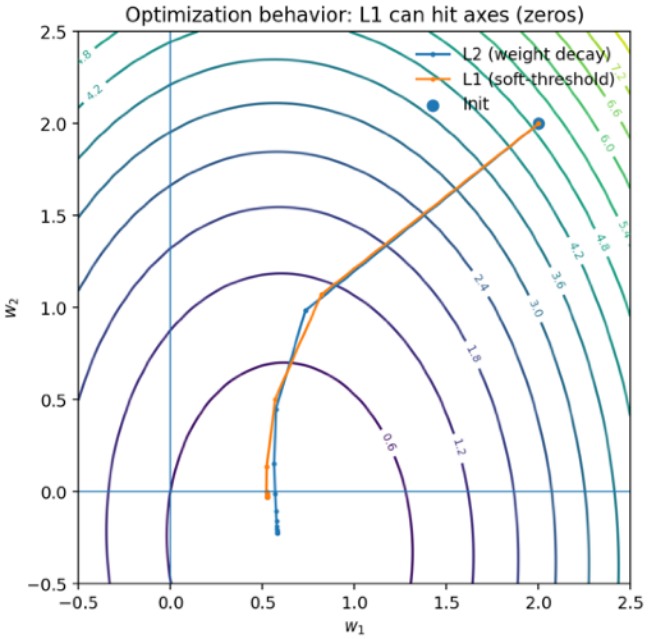
$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

L1 : promotes sparse weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

L2 : promotes small weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$



Regularization through weight decay

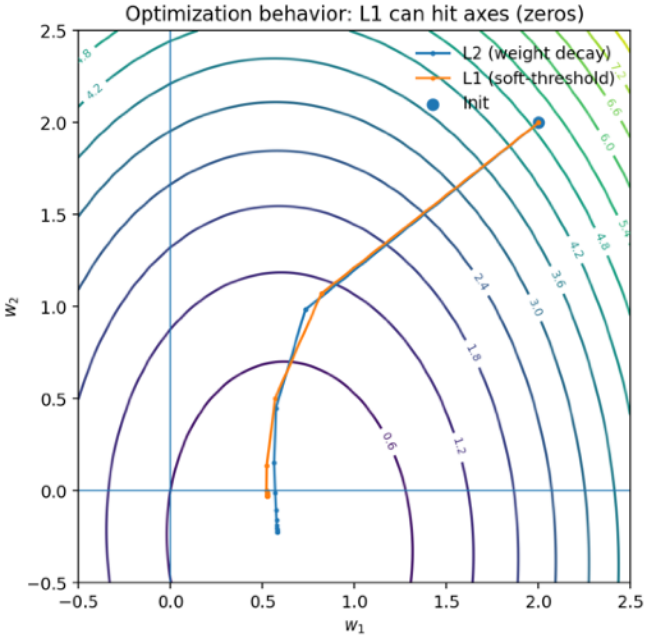
$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

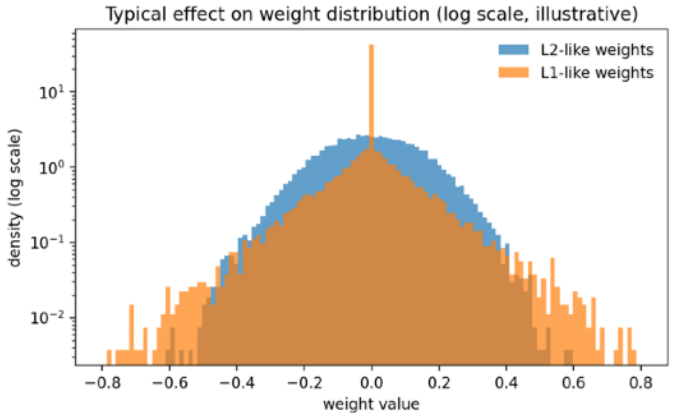
L1 : promotes sparse weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

L2 : promotes small weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$





Regularization through weight decay

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) \qquad (N = |\mathscr{D}|)$$

L1 : promotes sparse weights

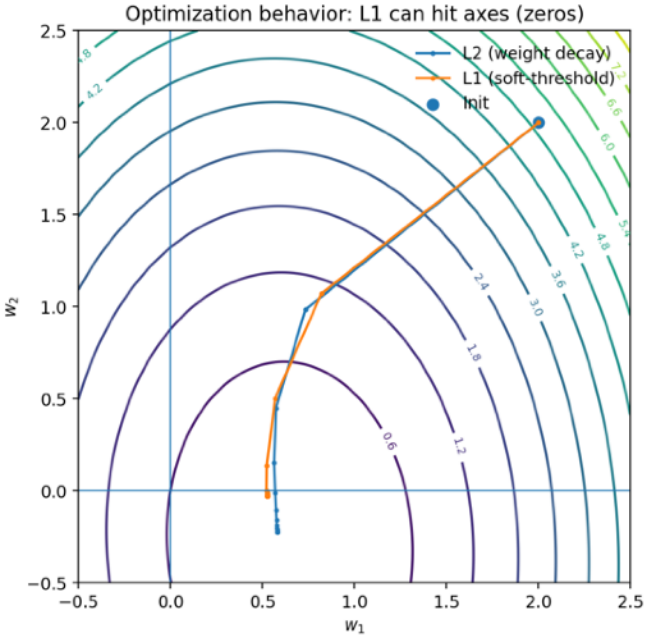
$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k|$$

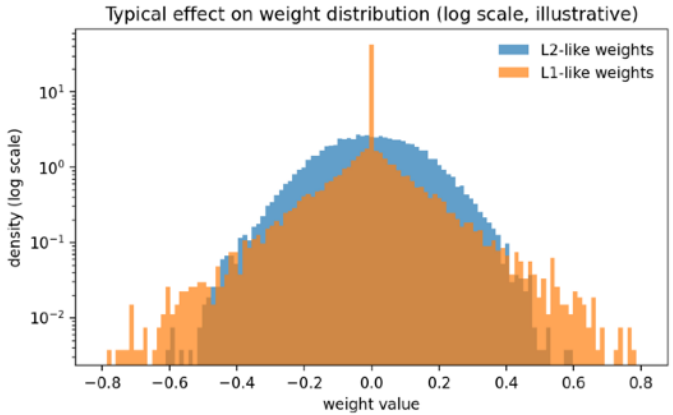
L2 : promotes small weights

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$

L1 and L2:

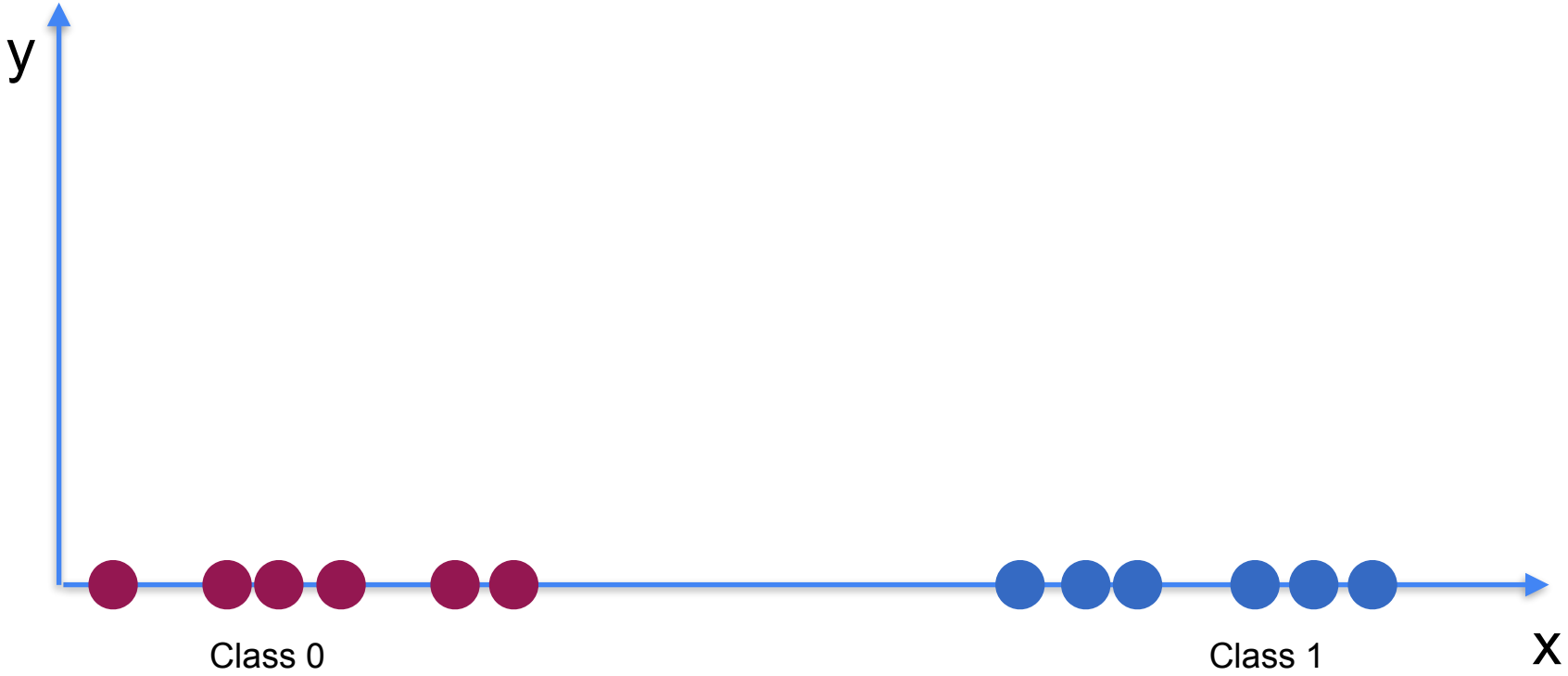
$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y})\in\mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \lambda_1 \sum_k |w_k| + \frac{\lambda_2}{2} \sum_k w_k^2$$





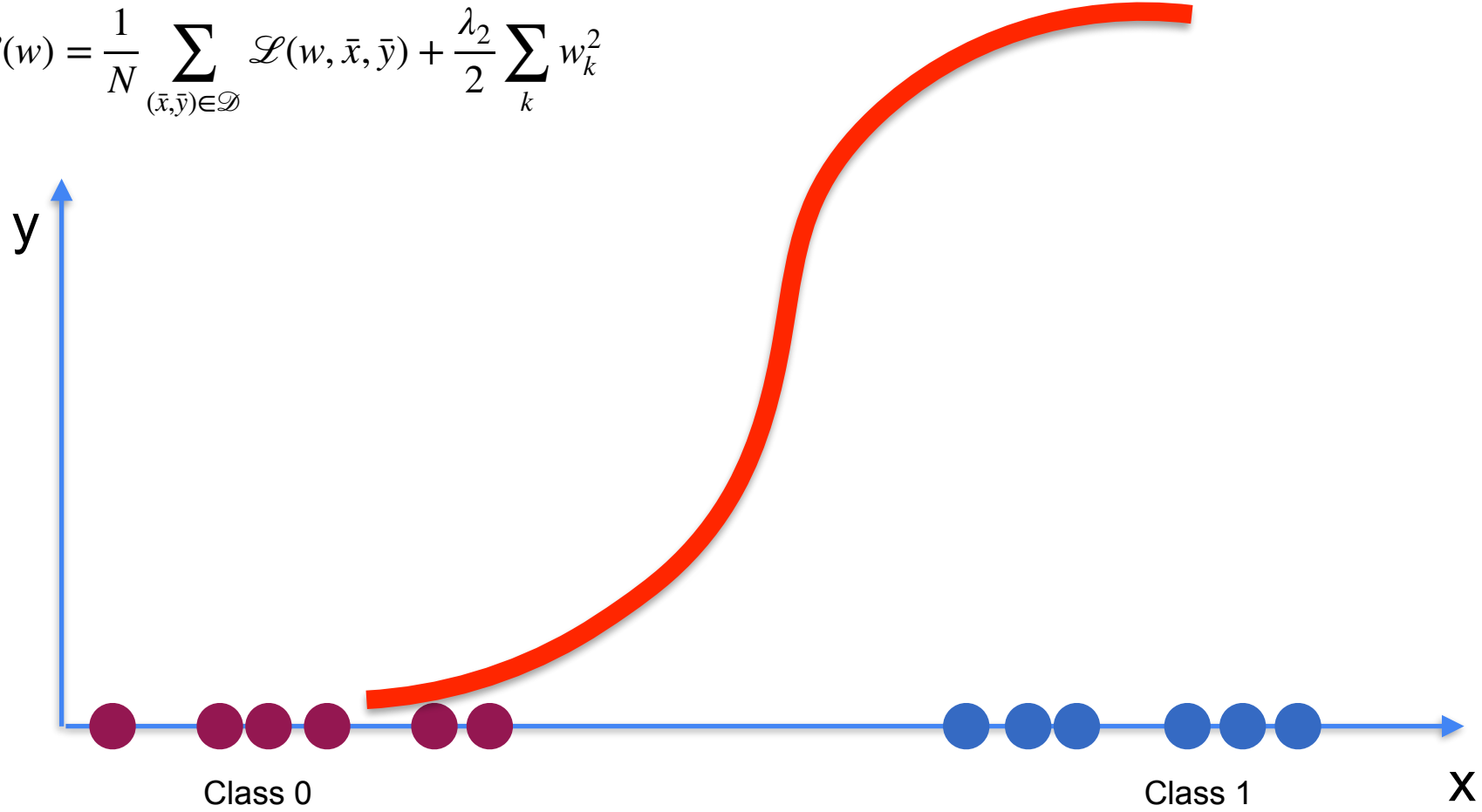
Another effect of L2 regularization: caution in decisions

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$



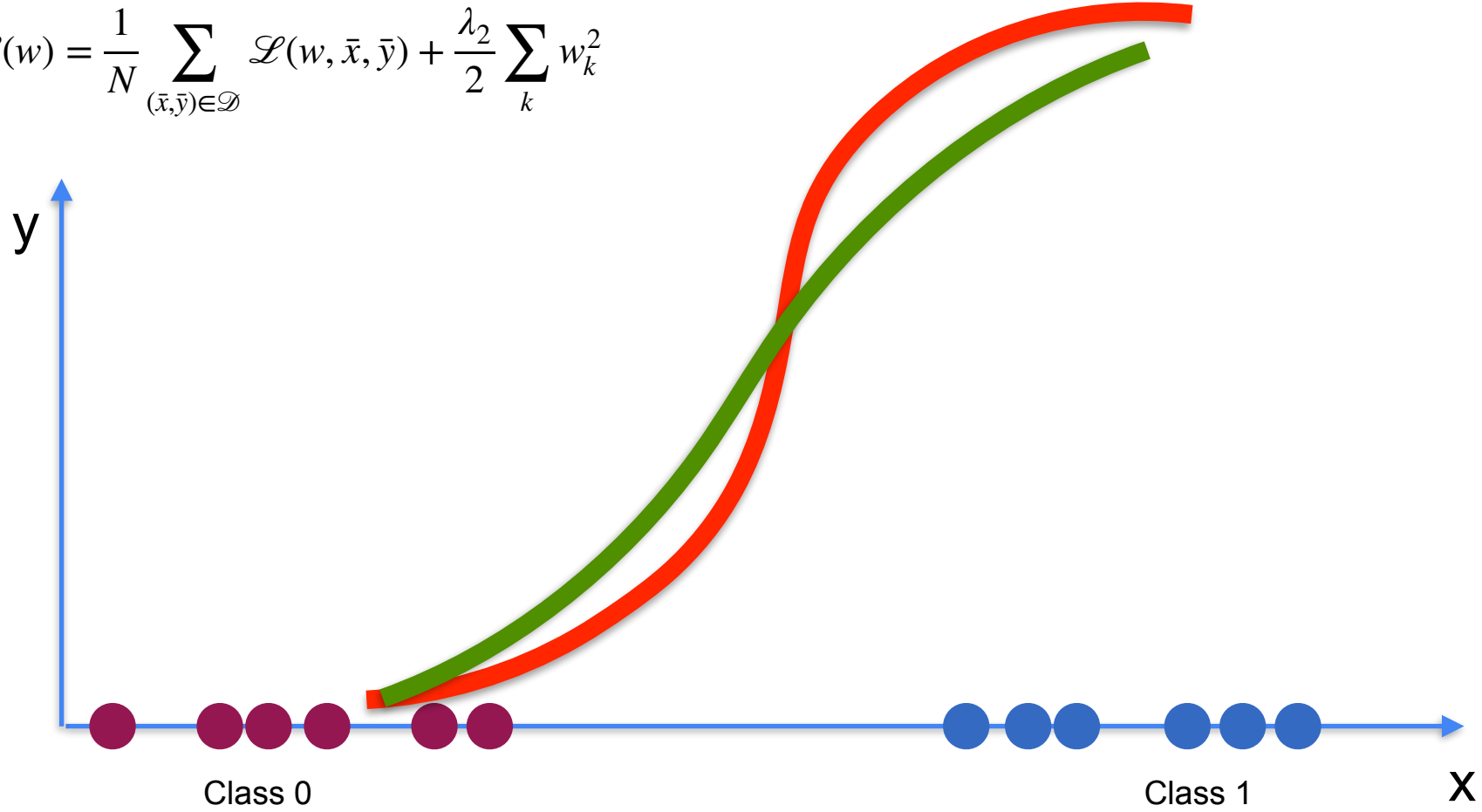
Another effect of L2 regularization: caution in decisions

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$



Another effect of L2 regularization: caution in decisions

$$\mathscr{L}(w) = \frac{1}{N} \sum_{(\bar{x},\bar{y}) \in \mathscr{D}} \mathscr{L}(w, \bar{x}, \bar{y}) + \frac{\lambda_2}{2} \sum_k w_k^2$$



Gradient clipping

$$g = |\nabla w|$$

Before applying backpropagation updates:

$$|g| > C \Rightarrow g \leftarrow \frac{Cg}{|g|}$$

Gradient clipping

$$g = |\nabla w|$$

$$|g| > C \Rightarrow g \leftarrow \frac{Cg}{|g|}$$

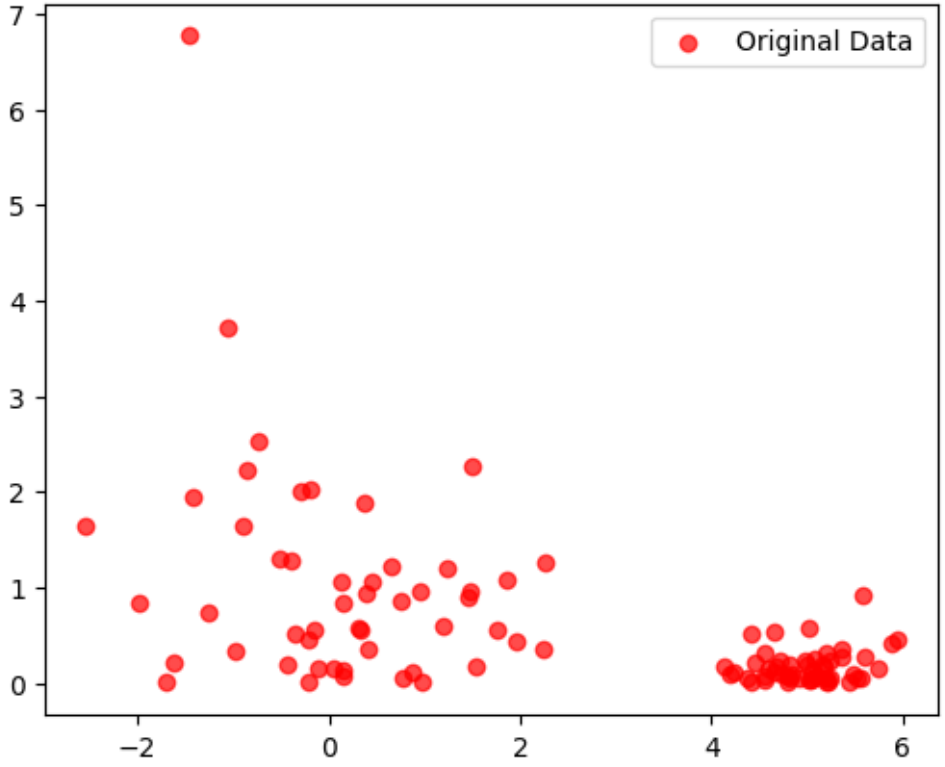
When is it helpful?

- Recurrent neural networks and long sequences
- Very deep networks
- Large weights

Data handling

Data normalization

Gradient descent can be trouble if the data spans very different ranges in different dimensions.



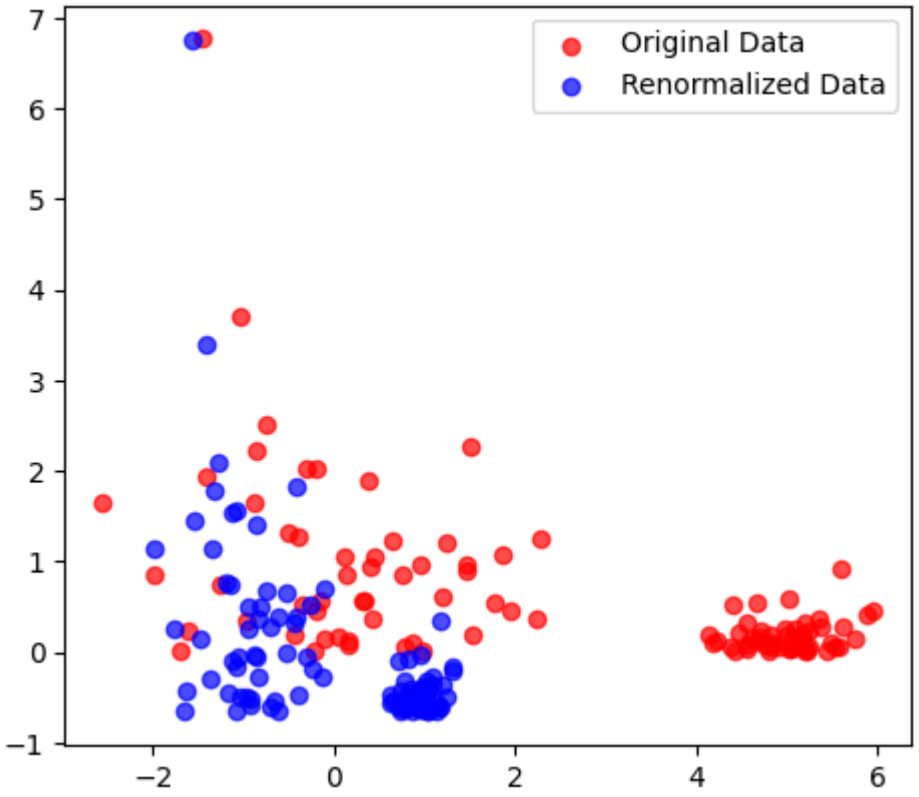
Data normalization

Gradient descent can be trouble if the data spans very different ranges in different dimensions.

$$\mu_i = \frac{1}{N} \sum_n x_{in}$$

$$\sigma_i^2 = \frac{1}{N} \sum_n (x_{ni} - \mu_i)^2$$

$$\bar{x_{ni}} = \frac{x_{ni} - \mu_i}{\sigma_i}$$



Since data normalization brings stability, shouldn't we do that for activations between layers?

$$\mu_i^{(k)} = \frac{1}{N} \sum_n x_{in}^{(k)}$$

$$\sigma_i^{(k)} = \sqrt{\frac{1}{N} \sum_n (x_{ni}^{(k)} - \mu_i^{(k)})^2}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_i^{(k)}}{\sigma_i^{(k)}}$$

Ioffe and Szegedy, 2015, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Since data normalization brings stability, shouldn't we do that for activations between layers?

$$\mu_i^{(k)} = \frac{1}{N} \sum_n x_{in}^{(k)}$$

$$\sigma_i^{(k)} = \sqrt{\frac{1}{N} \sum_n (x_{ni}^{(k)} - \mu_i^{(k)})^2}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_i^{(k)}}{\sigma_i^{(k)}}$$

Problems:

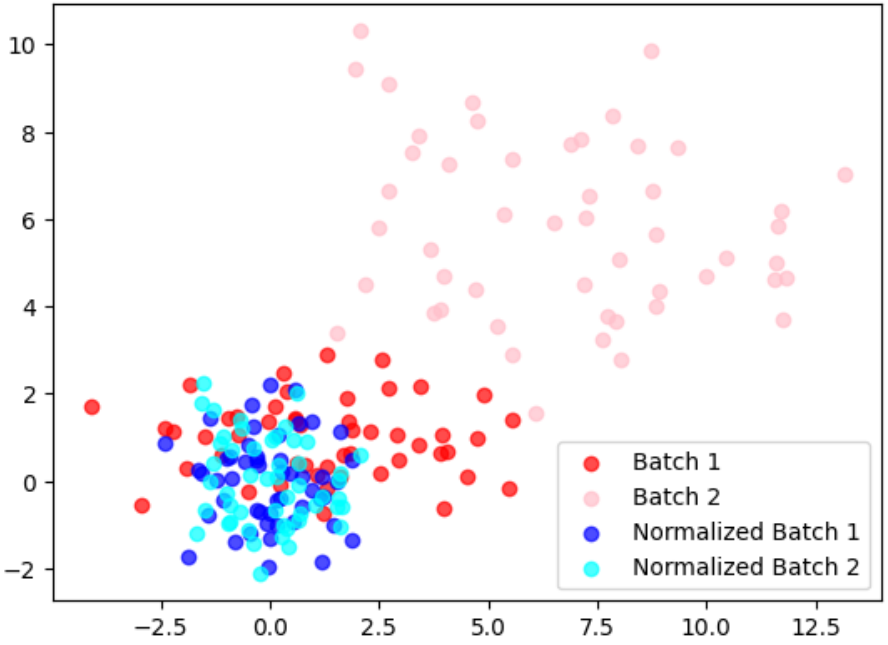
- It's not compatible with SGD
- Variance can be too small
- It removes degrees of freedom

Ioffe and Szegedy, 2015, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Batch normalization: training

Let's normalize by batch, and add some noise to the variance.

B = Batch size

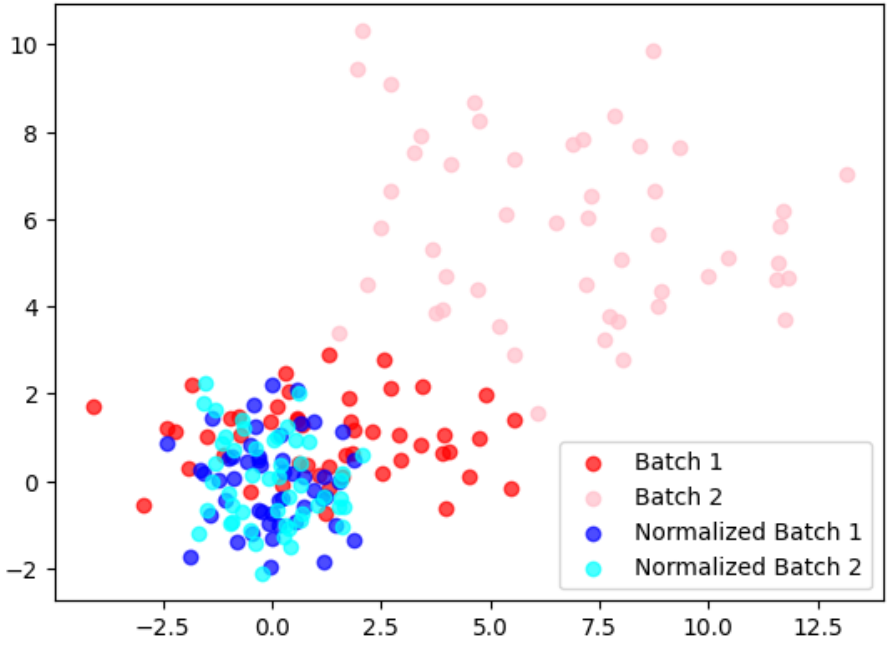


Batch normalization

Let's normalize by batch, and add some noise to the variance.

B = Batch size

$$\mu_{i,B}^{(k)} = \frac{1}{B} \sum_{n} x_{in}^{(k)}$$



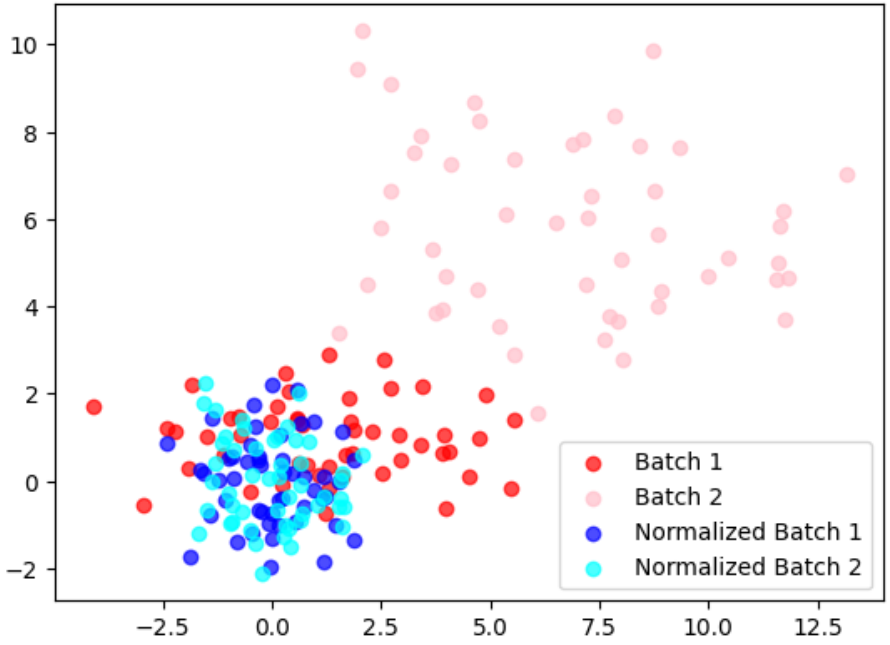
Batch normalization

Let's normalize by batch, and add some noise to the variance.

B = Batch size

$$\mu_{i,B}^{(k)} = \frac{1}{B}\sum_{n} x_{in}^{(k)}$$

$$\sigma_{i,B}^{(k)} = \sqrt{\frac{1}{B}\sum_{n}(x_{ni}^{(k)} - \mu_{i,B}^{(k)})^2 + \epsilon}$$



Batch normalization

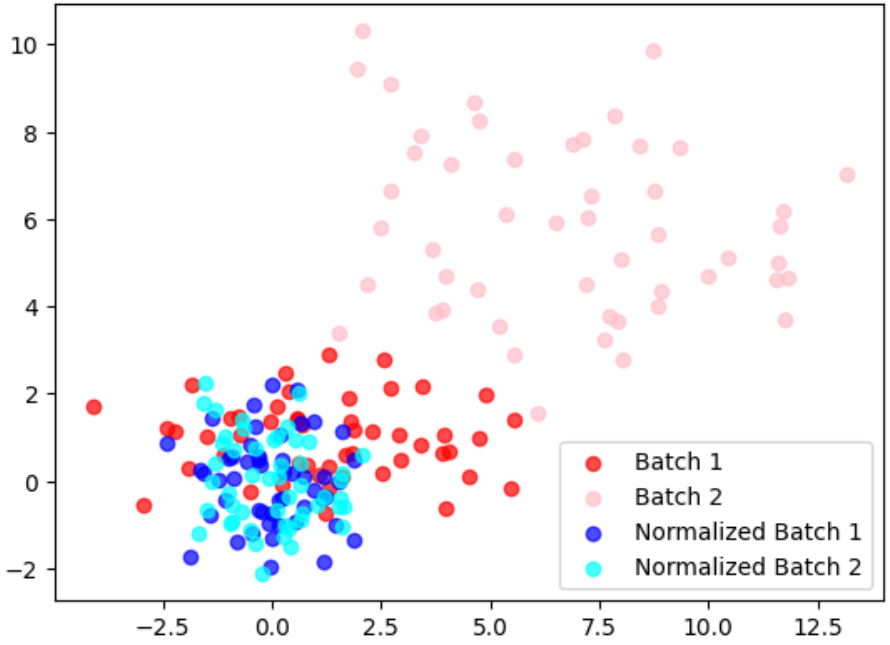
Let's normalize by batch, and add some noise to the variance.

B = Batch size

$$\mu_{i,B}^{(k)} = \frac{1}{B}\sum_{n} x_{in}^{(k)}$$

$$\sigma_{i,B}^{(k)} = \sqrt{\frac{1}{B}\sum_{n}(x_{ni}^{(k)} - \mu_{i,B}^{(k)})^2 + \epsilon}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_{i,B}^{(k)}}{\sigma_{i,B}^{(k)}}$$



Batch normalization

Then give the degrees of freedom back, through learned scale and shift

$$\mu_{i,B}^{(k)} = \frac{1}{B}\sum_{n} x_{in}^{(k)}$$

$$\sigma_{i,B}^{(k)} = \sqrt{\frac{1}{B}\sum_{n}(x_{ni}^{(k)} - \mu_{i,B}^{(k)})^2 + \epsilon}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_{i,B}^{(k)}}{\sigma_{i,B}^{(k)}}$$

$$\hat{x}_{ni}^{(k)} = \boxed{\gamma_i}\bar{x}_{ni}^{(k)} + \boxed{\beta_i}$$

Batch normalization

Then give the degrees of freedom back, through learned scale and shift

$$\mu_{i,B}^{(k)} = \frac{1}{B}\sum_{n} x_{in}^{(k)}$$

$$\sigma_{i,B}^{(k)} = \sqrt{\frac{1}{B}\sum_{n}(x_{ni}^{(k)} - \mu_{i,B}^{(k)})^2 + \epsilon}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_{i,B}^{(k)}}{\sigma_{i,B}^{(k)}}$$

$$\hat{x}_{ni}^{(k)} = \boxed{\gamma_i}\bar{x}_{ni}^{(k)} + \boxed{\beta_i}$$

Scale and shift are learned across batches.

SGD computational advantages are safe, degrees of freedoms are preserved, and stability is increased.

Batch normalization

Can’t do stuff in batches at inference time! We can compute global averages across all training data, with latest parameters ...

$$\mu_i^{(k)} = \frac{1}{N} \sum_n x_{in}^{(k)}$$

$$\sigma_i^{(k)} = \sqrt{\frac{1}{N} \sum_n (x_{ni}^{(k)} - \mu_i^{(k)})^2}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_i^{(k)}}{\sigma_i^{(k)}}$$

Batch normalization: inference

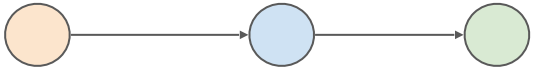
Can’t do stuff in batches at inference time! We can compute global averages across all training data, with latest parameters …
…or we can **maintain moving averages** per layer during training and store them with the parameters

$$\hat{\mu_i^{(k)}} = \alpha \hat{\mu_i^{(k)}} + (1 - \alpha)\mu_i^{(k)}$$

$$\hat{\sigma_i^{(k)}} = \alpha \hat{\sigma_i^{(k)}} + (1 - \alpha)\sigma_i^{(k)}$$

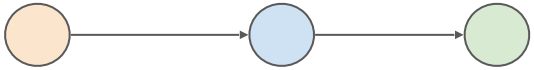
Batch normalization: before or after nonlinearity ?

$$x \xrightarrow{w,b} s \xrightarrow{\sigma} x$$

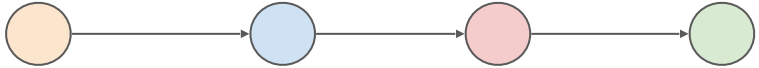


Batch normalization: before or after nonlinearity ?

$$x \xrightarrow{w,b} s \xrightarrow{\sigma} x$$



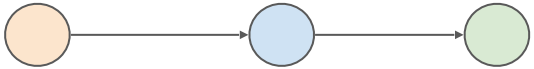
$$x \xrightarrow{w,b} s \xrightarrow{\gamma,\beta,\mu,\sigma} \bar{s} \xrightarrow{\sigma} \bar{x}$$



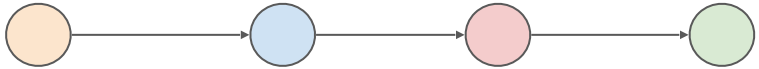
Original formulation.
Motivation: Better batch-normalize before nonlinearity compresses some of the signal.

Batch normalization: before or after nonlinearity ?

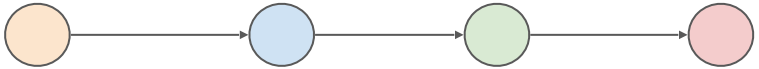
$$x \xrightarrow{w,b} s \xrightarrow{\sigma} x$$



$$x \xrightarrow{w,b} s \xrightarrow{\gamma,\beta,\mu,\sigma} \bar{s} \xrightarrow{\sigma} \bar{x}$$



$$x \xrightarrow{w,b} s \xrightarrow{\sigma} x \xrightarrow{\gamma,\beta,\mu,\sigma} \bar{x}$$



Original formulation.
Motivation: Better batch-normalize before nonlinearity compresses some of the signal.

Works better in practice.
Counter-argument: Nonlinearity does not compress towards normally distributed.

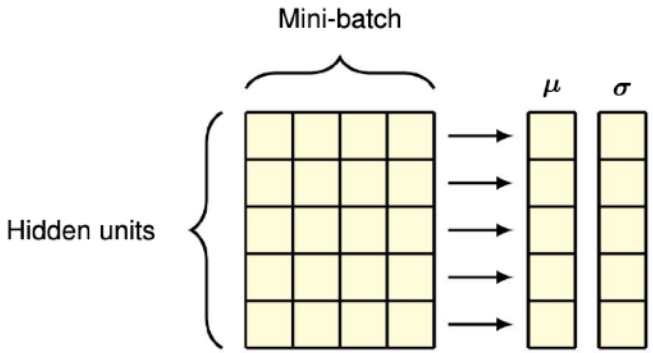
Instead of normalizing for each unit across all points in a batch…

$$\mu_{i,B}^{(k)} = \frac{1}{B}\sum_{n} x_{ni}^{(k)}$$

$$\sigma_{i,B}^{(k)} = \sqrt{\frac{1}{B}\sum_{n}(x_{ni}^{(k)} - \mu_{i,B}^{(k)})^2 + \epsilon}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_{i,B}^{(k)}}{\sigma_{i,B}^{(k)}}$$

$$\hat{x}_{ni}^{(k)} = \gamma_i \bar{x}_{ni}^{(k)} + \beta_i$$



Layer normalization

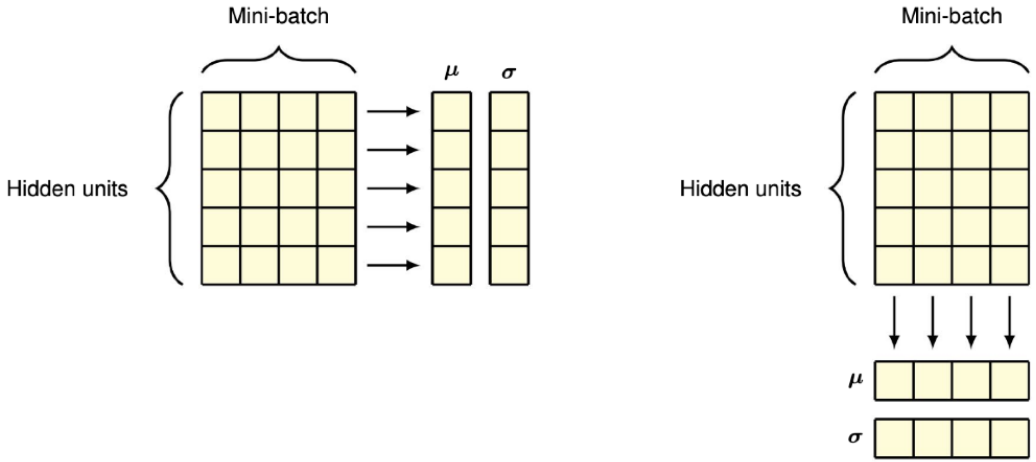
Instead of normalizing for each unit across all points in a batch…
…we can normalize for each point across all units

$$\mu_{i,n}^{(\mathcal{L})} = \frac{1}{L}\sum_{k=1}^{L} x_{ni}^{(k)}$$

$$\sigma_{i,n}^{(\mathcal{L})} = \sqrt{\frac{1}{L}\sum_{k}(x_{ni}^{(k)} - \mu_{i,n}^{(\mathcal{L})})^2 + \epsilon}$$

$$\bar{x}_{ni}^{(k)} = \frac{x_{ni}^{(k)} - \mu_{i,n}^{(\mathcal{L})}}{\sigma_{i,n}^{(\mathcal{L})}}$$

$$\hat{x}_{ni}^{(k)} = \gamma_i^{(\mathcal{L})}\bar{x}_{ni}^{(k)} + \beta_i^{(\mathcal{L})}$$



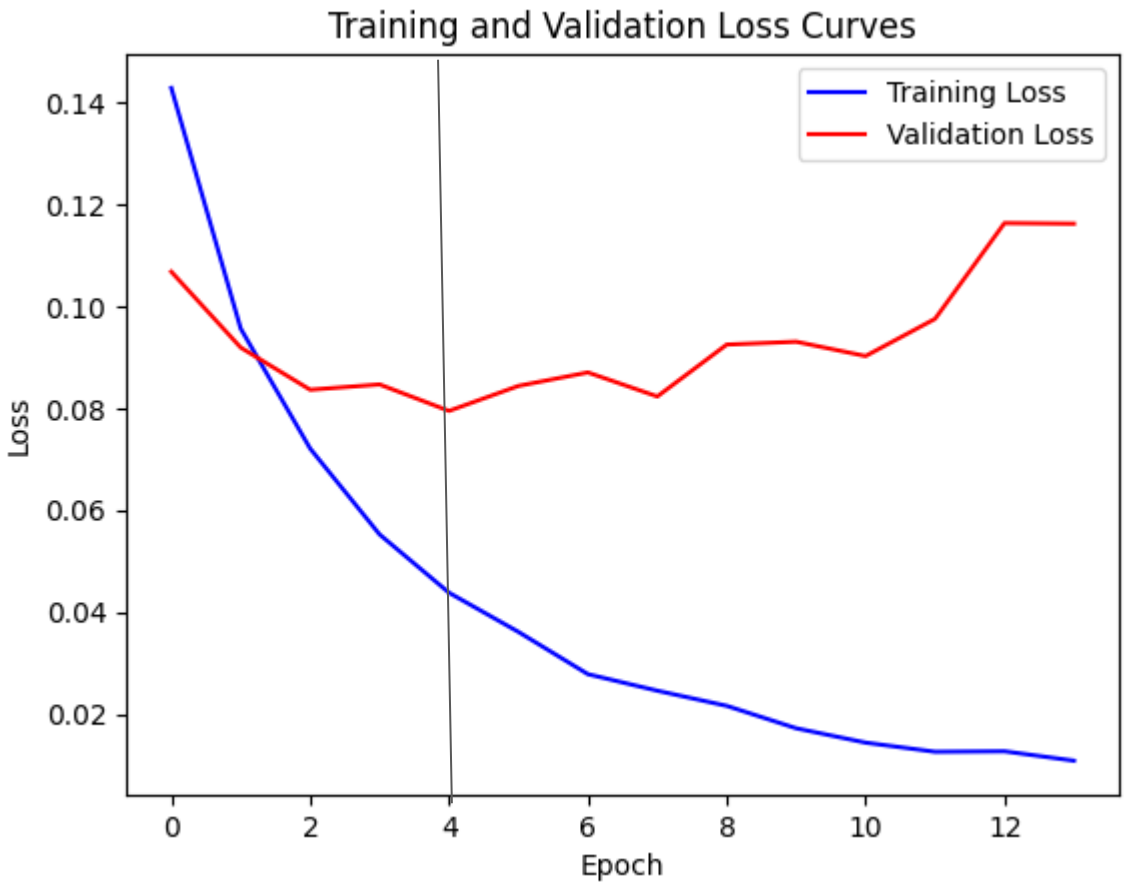
www.bishopbook.com

Normalizing hidden units

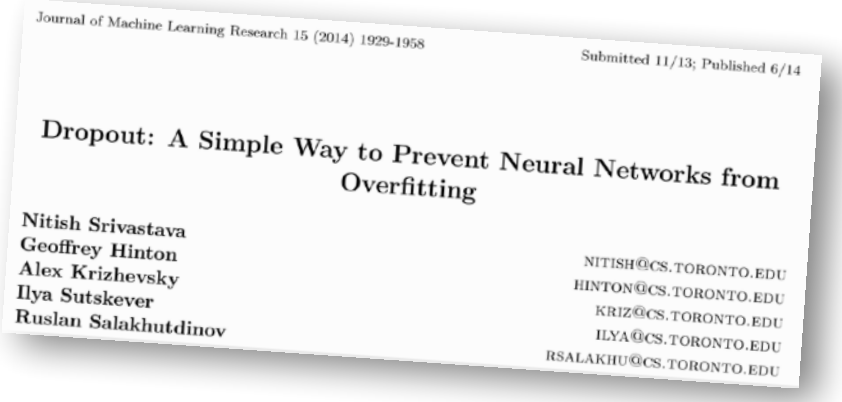
Batch normalization By batch, for each unit	**Layer normalization** By unit in a layer, for each point
Suitable for large batches	Works better for small batches
More memory (requires to store mean and variance across the mini batch)	Less memory
Empirically, it works better for feedforward neural networks	Empirically, it works better for recurrent and convolutional networks, as well as transformers

Training algorithm

Early stopping



Dropout

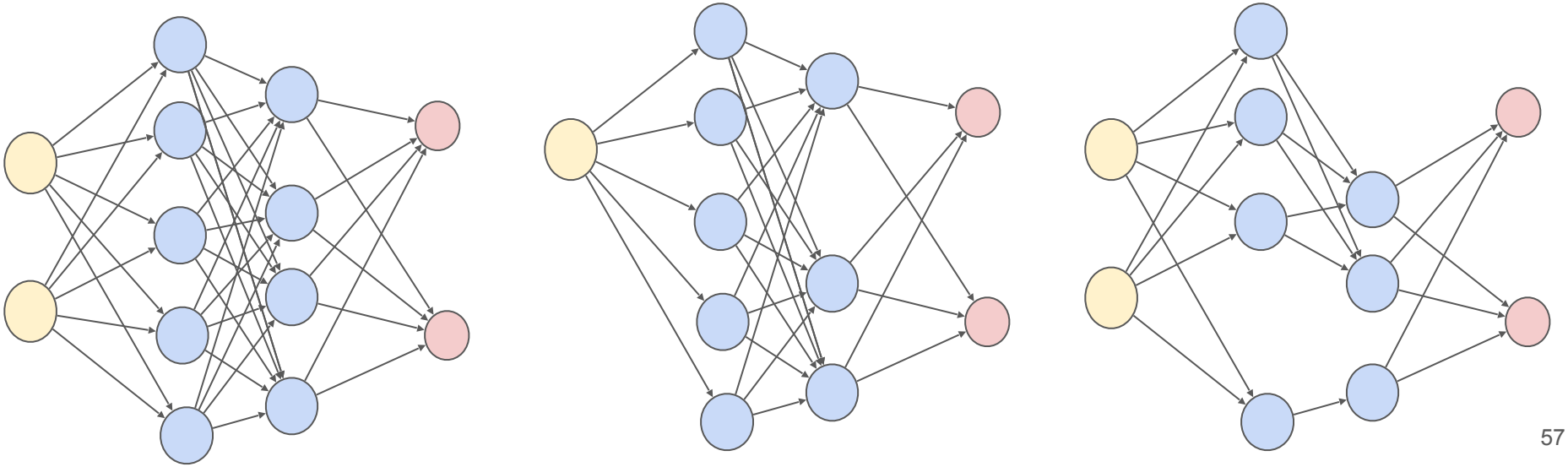

Journal of Machine Learning Research 15 (2014) 1929-1958

Submitted 11/13; Published 6/14

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

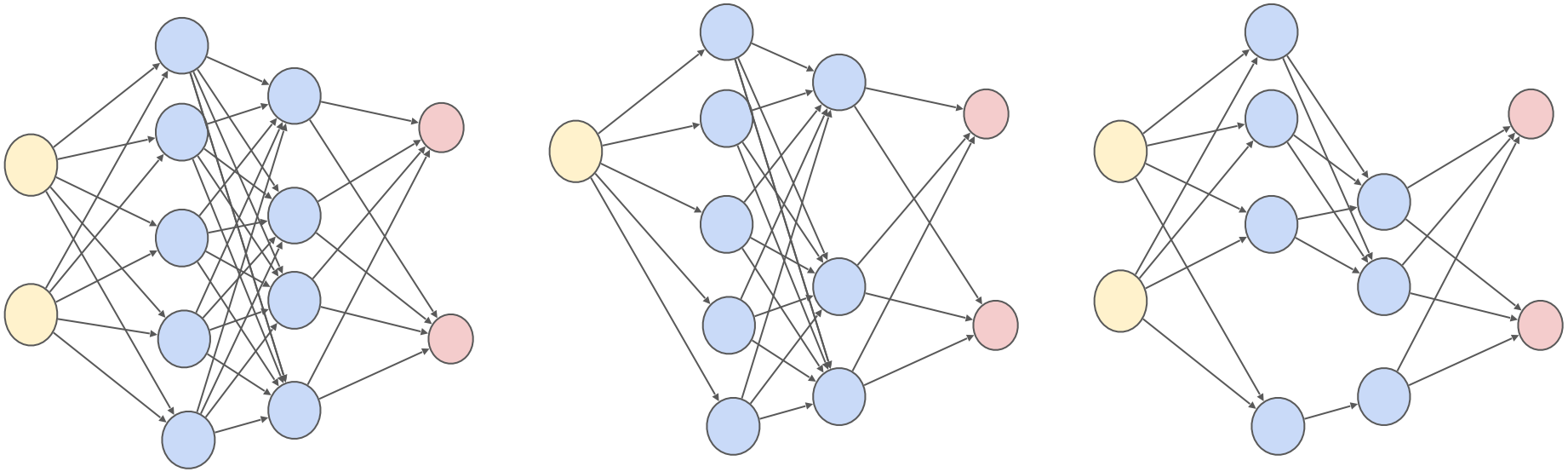
Nitish Srivastava NITISH@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU
Alex Krizhevsky KRIZ@CS.TORONTO.EDU
Ilya Sutskever ILYA@CS.TORONTO.EDU
Ruslan Salakhutdinov RSALAKHU@CS.TORONTO.EDU

At training time: randomly drop nodes in computation (zero-out activations) with probability p



Dropout

At inference time: average out over all training “masks”, or do not mask at all



Discourages overfitting, and overly relying on individual nodes

Hyperparameter tuning

We have already seen a lot of hyperparameters… we can't try them all!

$$f : \chi \rightarrow \mathbb{R}$$

Configuration space

Validation Error

It's hard to optimize. So let's just do our best when we search.

Hyperparameter configuration space

List the parameters that are important.
Determine their type and reasonable range.
VERY IMPORTANT: Is log-scale more appropriate than linear?

https://d2l.ai/chapter_hyperparameter-optimization/hyperopt-intro.html

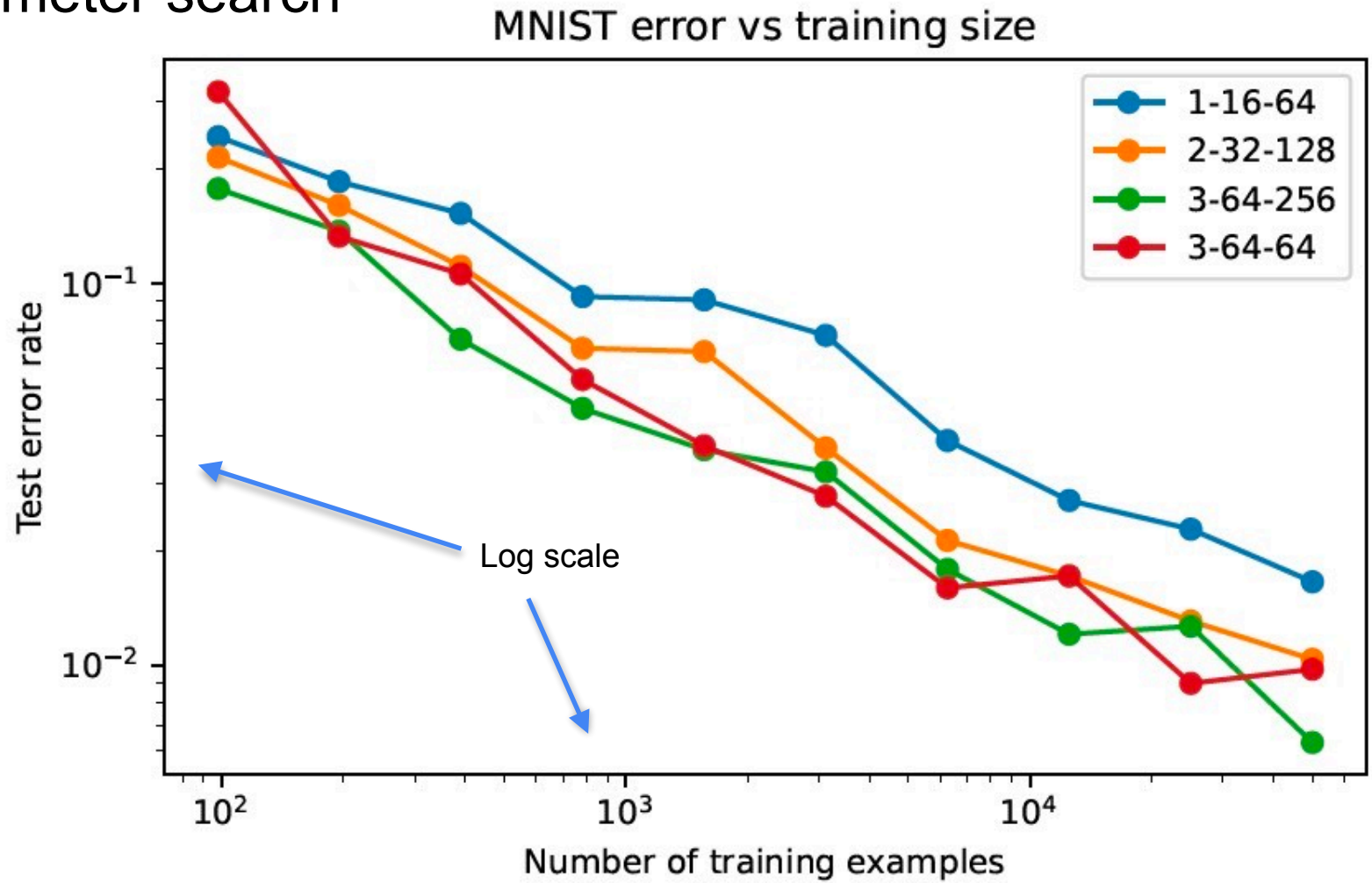
Hyperparameter configuration space

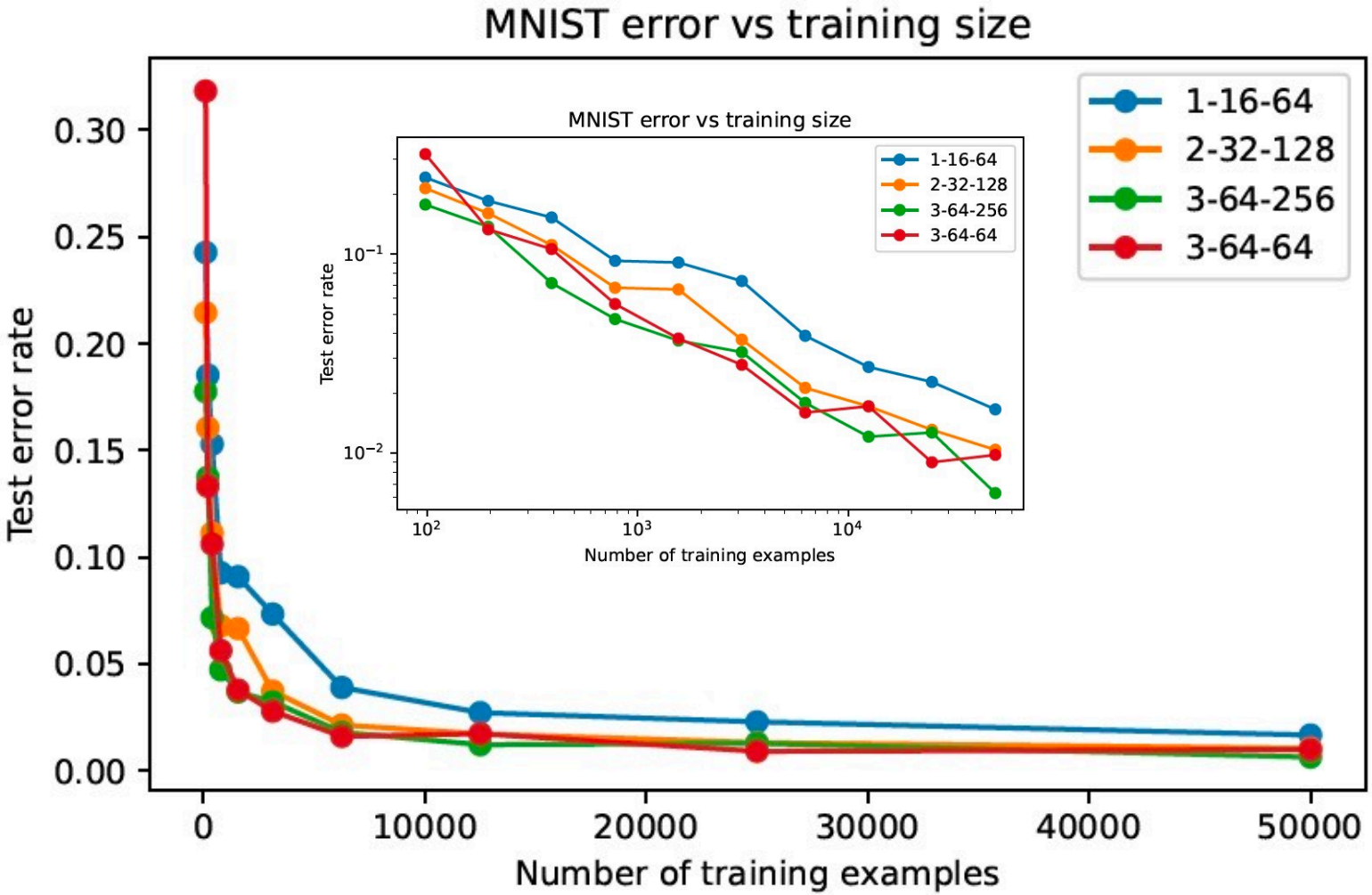
List the parameters that are important.
Determine their type and reasonable range.
VERY IMPORTANT: Is log-scale more appropriate than linear?

Name	Type	Hyperparameter Ranges	log-scale
learning rate	float	:math:` [10^{-6},10^{-1}]`	yes
batch size	integer	$[8, 256]$	yes
momentum	float	$[0, 0.99]$	no
activation function	categorical	:mat h:*{textrm{tanh} , textrm{relu}}*	•
number of units	integer	$[32, 1024]$	yes
number of layers	integer	$[1, 6]$	no

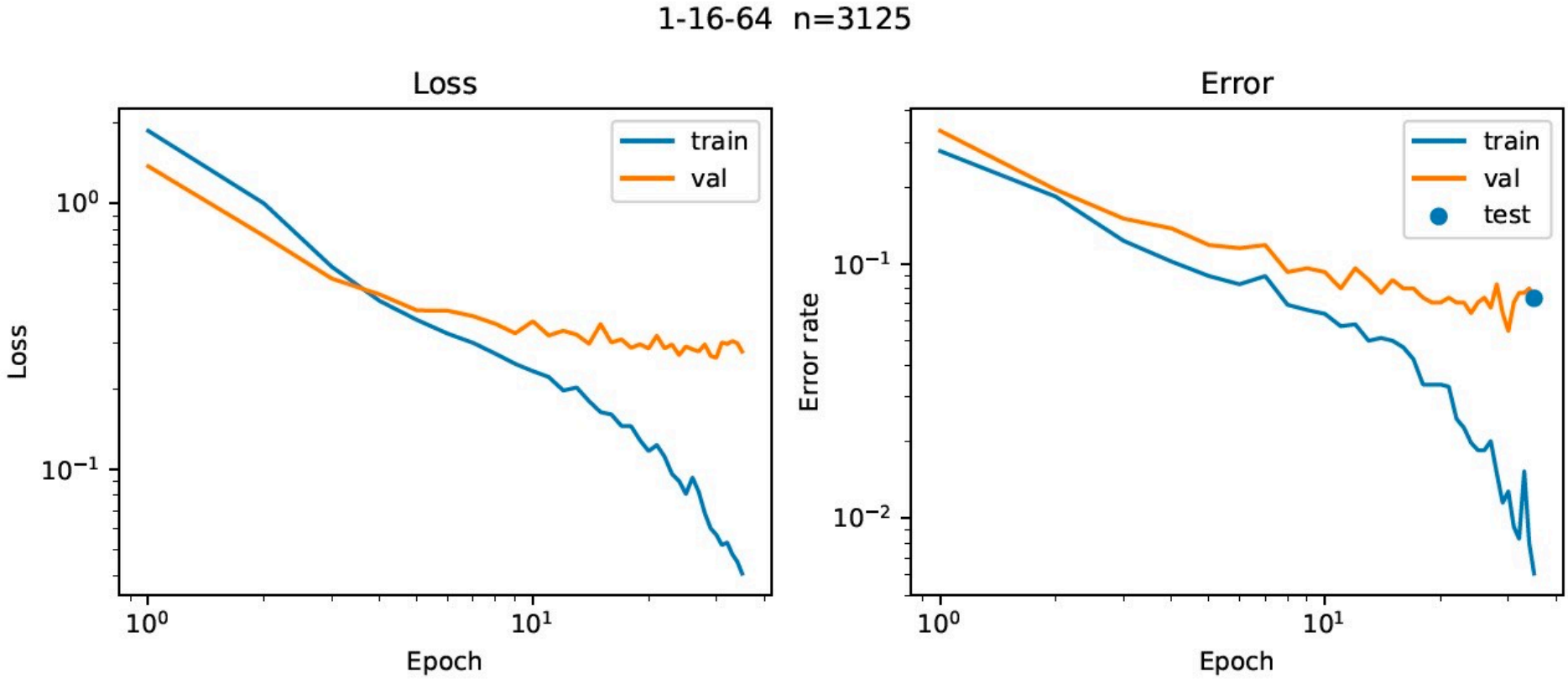
https://d2l.ai/chapter_hyperparameter-optimization/hyperopt-intro.html

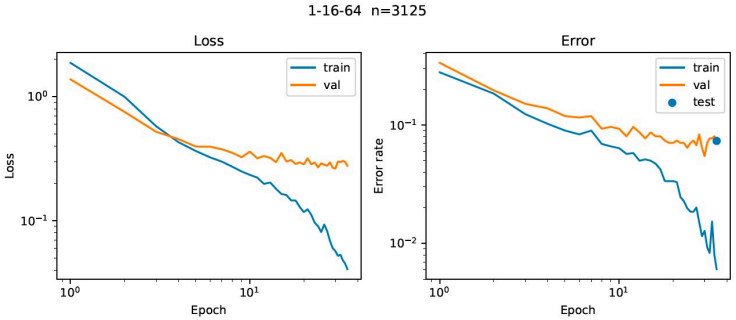
Hyperparameter search

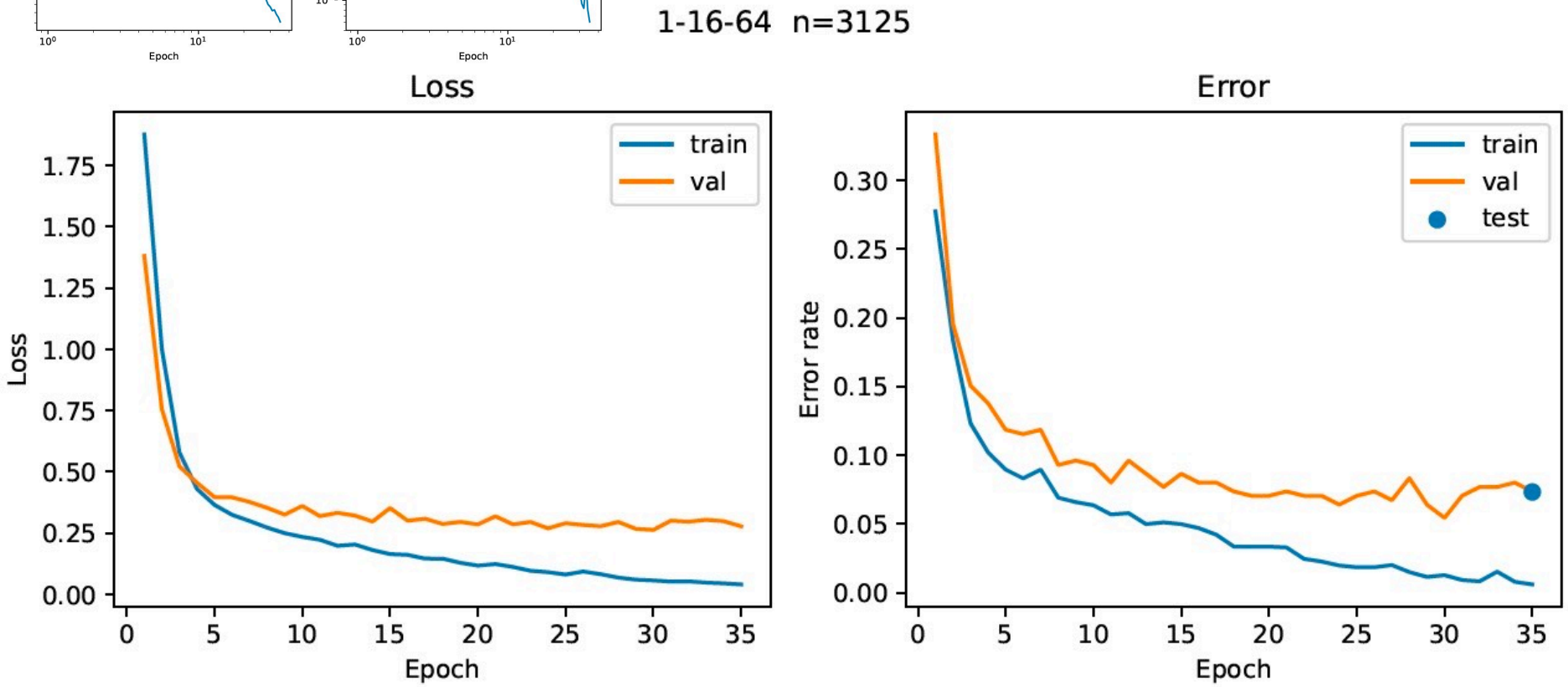



MNIST error vs training size
Test error rate
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
10000
20000
30000
40000
50000
Number of training examples
1-16-64
2-32-128
3-64-256
3-64-64
MNIST error vs training size
Test error rate
10^{-1}
10^{-2}
10^{2}
10^{3}
10^{4}
Number of training examples
1-16-64
2-32-128
3-64-256
3-64-64

Always check your learning curves!




1-16-64 n=3125
Loss
train
val
Loss
Epoch
Error
train
val
test
Error rate
Epoch


1-16-64 n=3125
Loss
train
val
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Loss
0
5
10
15
20
25
30
35
Epoch
Error
train
val
test
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Error rate
0
5
10
15
20
25
30
35
Epoch

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point
- Random Search - independent samples from configuration space
  - Not tied to a grid, we can explore more

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point
- Random Search - independent samples from configuration space
  - Not tied to a grid, we can explore more
- Asynchronous Random Search - schedule a new run as soon as a slot opens
  - Different runs can take wildly different times

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point
- Random Search - independent samples from configuration space
  - Not tied to a grid, we can explore more
- Asynchronous Random Search - schedule a new run as soon as a slot opens
  - Different runs can take wildly different times
- Bayesian Optimization - based on “surrogate” probabilistic model of f
  - Explore/exploit strategy to determine new points to evaluate

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point
- Random Search - independent samples from configuration space
  - Not tied to a grid, we can explore more
- Asynchronous Random Search - schedule a new run as soon as a slot opens
  - Different runs can take wildly different times
- Bayesian Optimization - based on “surrogate” probabilistic model of f
  - Explore/exploit strategy to determine new points to evaluate
- Multifidelity and hyperband - allocates different resources according to how promising the runs look
  - For example, every n iterations discard the m less promising runs

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Hyperparameter search strategies

- Grid Search - exhaustively explore configuration space on a grid
  - Can run in parallel. Make sure to determine stopping point
- Random Search - independent samples from configuration space
  - Not tied to a grid, we can explore more
- Asynchronous Random Search - schedule a new run as soon as a slot opens
  - Different runs can take wildly different times
- Bayesian Optimization - based on “surrogate” probabilistic model of f
  - Explore/exploit strategy to determine new points to evaluate
- Multifidelity and hyperband - allocates different resources according to how promising the runs look
  - For example, every n iterations discard the m less promising runs
- Evolutionary optimization - mutation or crossover between “parents”
  - Iterative procedure, parents are the best

https://wires.onlinelibrary.wiley.com/doi/epdf/10.1002/widm.1484

Other techniques we'll talk about later

- Parameter sharing
- Residual connection
- Data Augmentation
- Invariant features
- ….

Recap

Convergence

Good fit and generalization

Recap and next time

- When training, we need to worry about convergence, good fit and generalization
- Many techniques are available, that help with one or all of these concerns
- We went over the most commonly used ones
- We highlighted the importance to combine them effectively, which often boils down to hyperparameter search

Next time:

- Architectures, modules, layers